

Neural networks, machine learning and randomness

Preliminaries:
recalling what you already know

Optional course for master students

Martin Holeňa

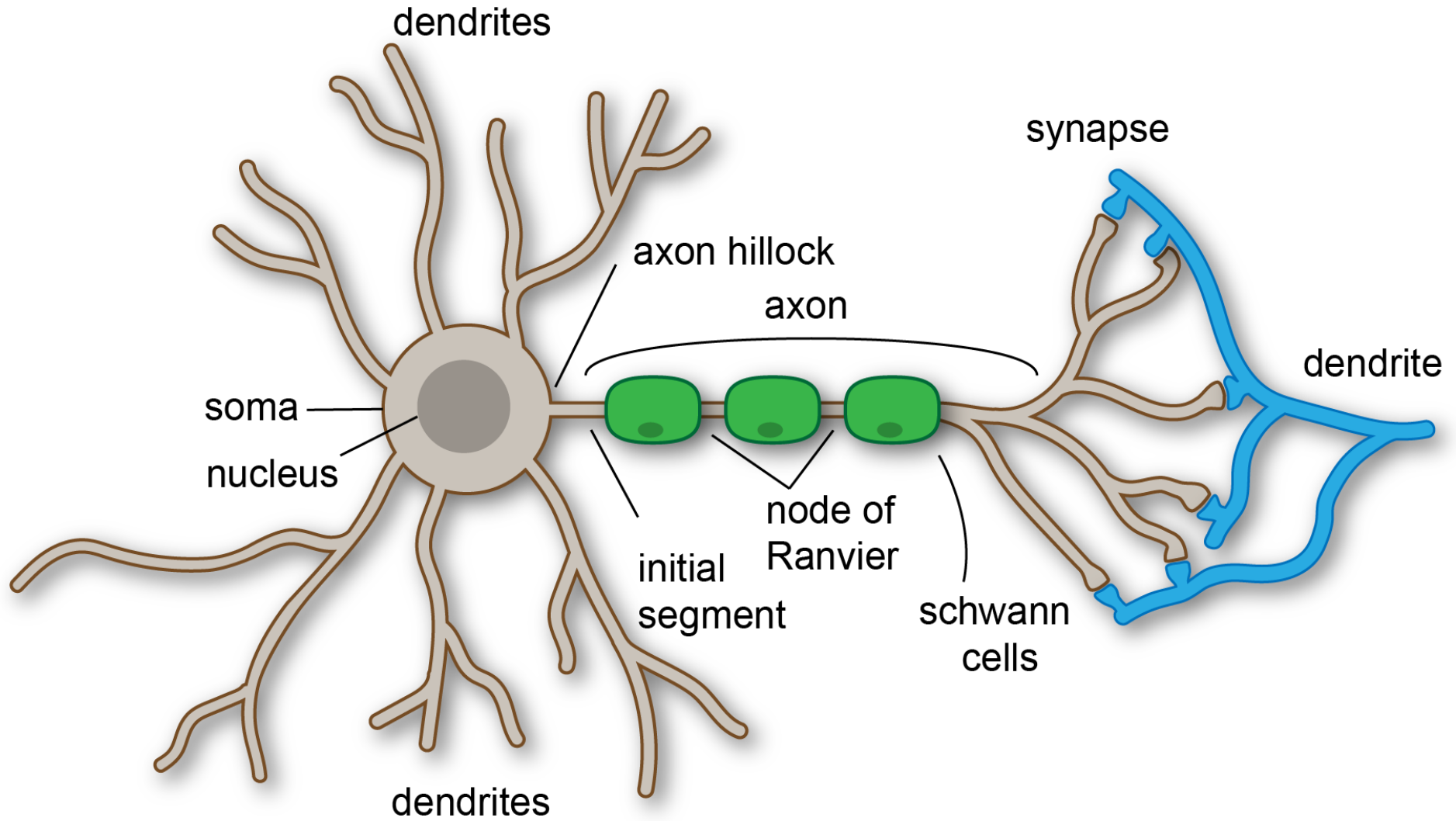


What will it be about?

- ◆ Basic concepts concerning artificial neural networks (ANNs)
- ◆ Most important kinds of artificial neural networks
- ◆ Basic concepts pertaining to machine learning (ML)
- ◆ Main kinds of machine learning methods
- ◆ Most important concepts pertaining to randomness

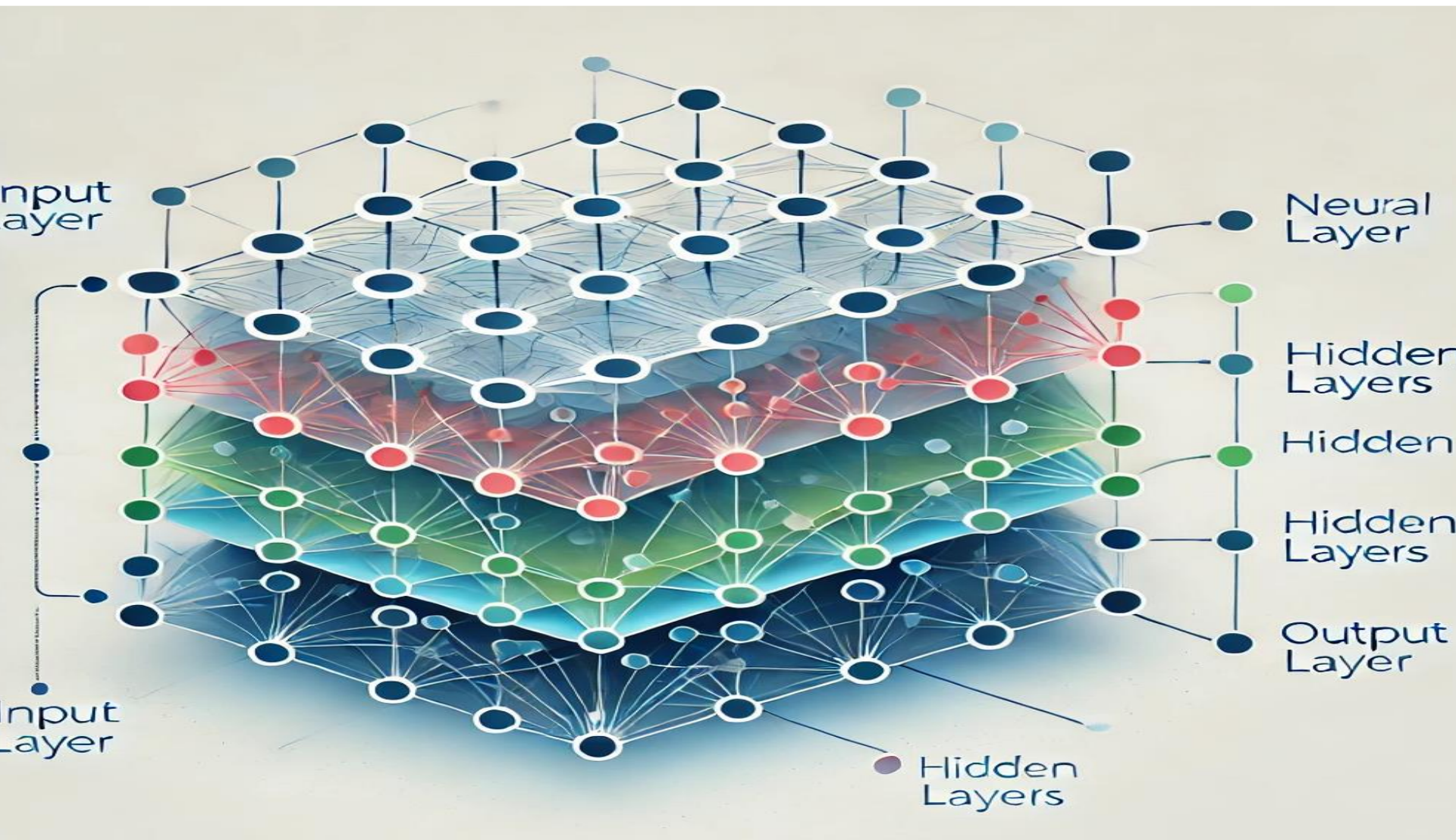
Neurons and connections

- ◆ Neurons – fundamental units of ANNs, analogous to biological
 - somatic operation: process incoming signals, send them further
- ◆ Connections – synaptic operation: weighting signals between neurons
- ◆ *Structure* of a neuron: *weights, bias, activation function*
 - example activation functions: logistic $f(z) = \frac{1}{1+e^{-z}}$, hyperbolic tangent $f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, rectified linear unit $f(z) = \max(0, z)$



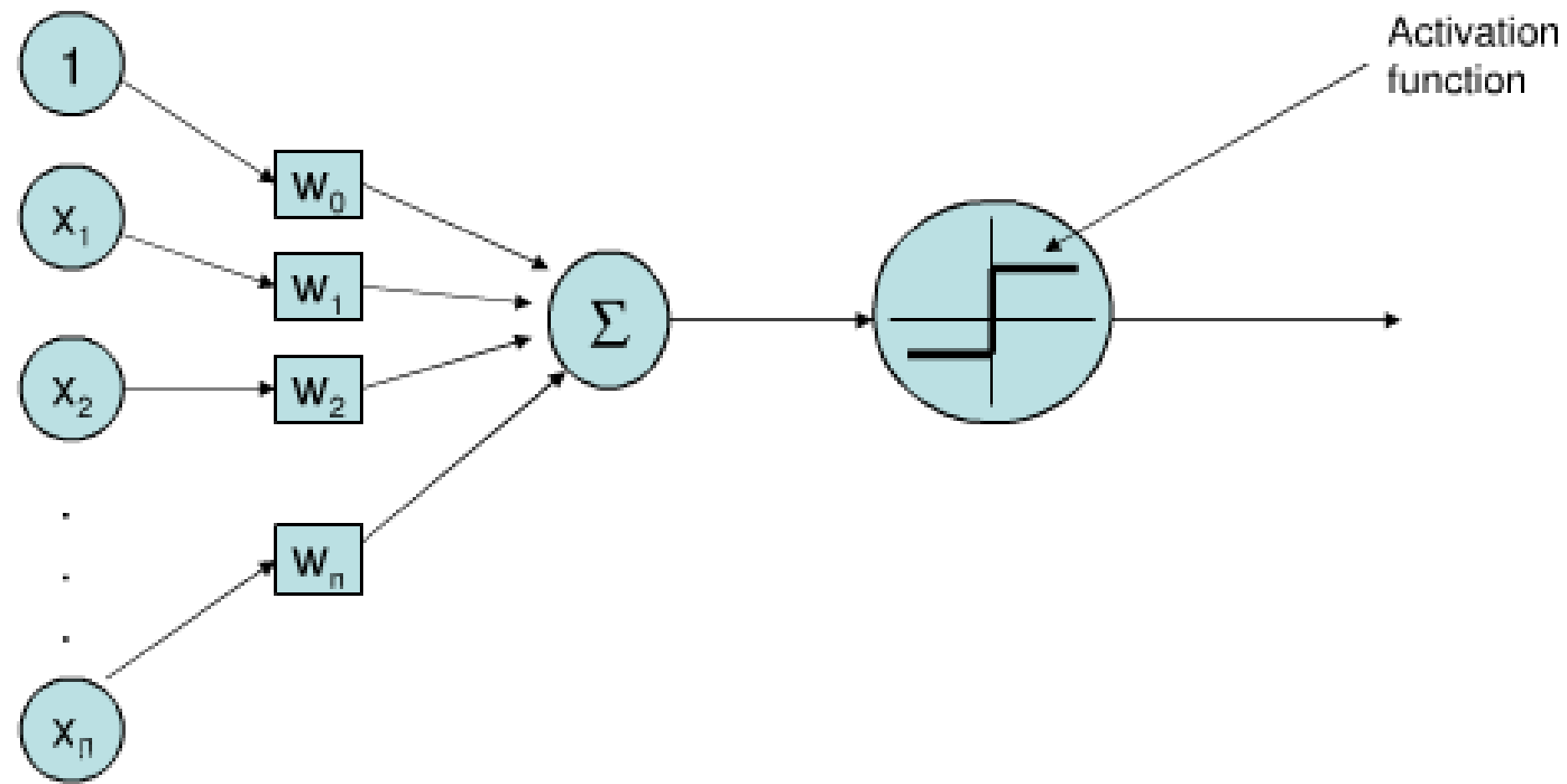
Neural network architecture

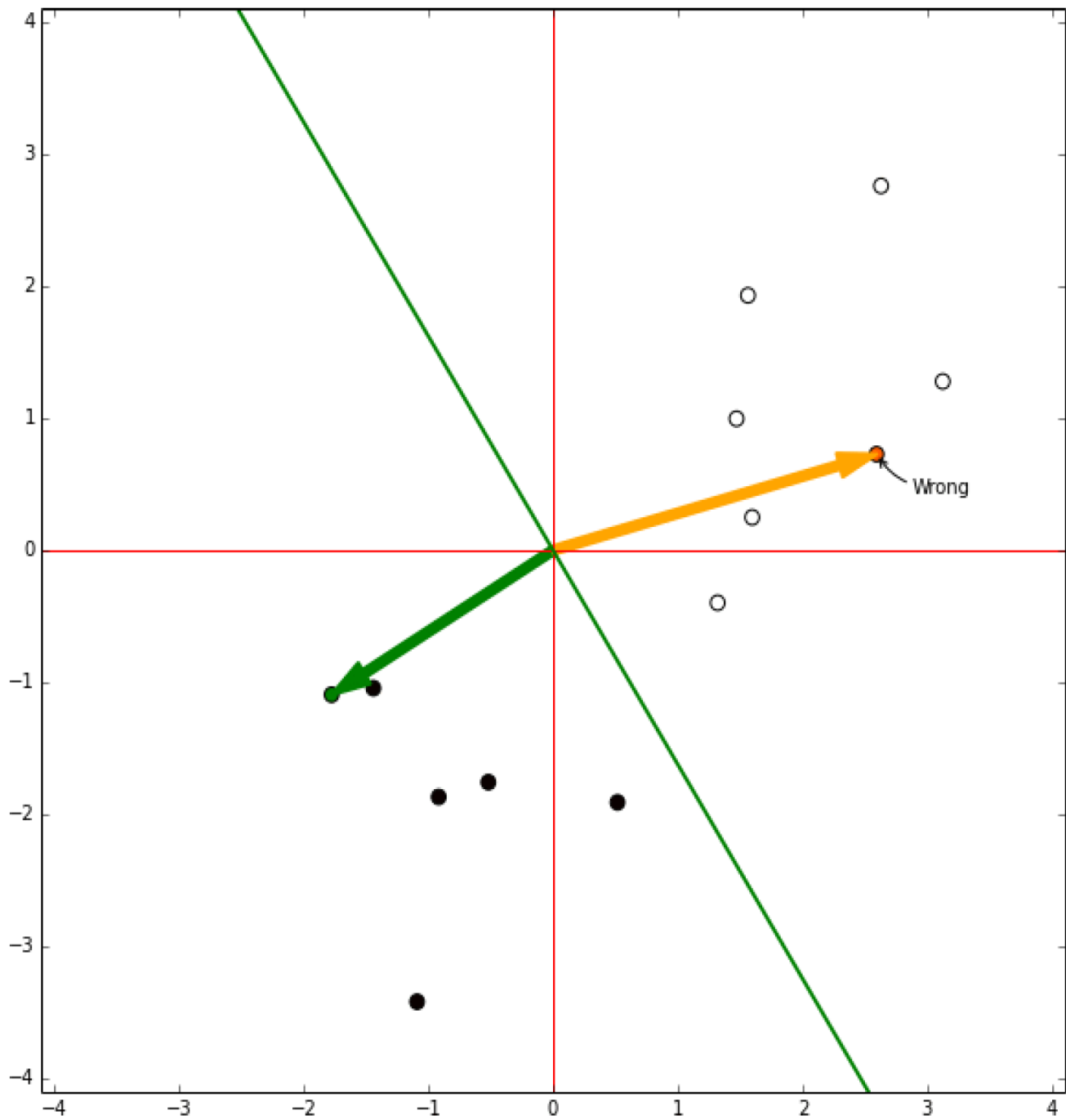
- ◆ Architecture: a specific pattern of connections between neurons
- ◆ *Layered* architecture: a linearly ordered sequence of layers
 - disjoint, signal transmitted only in the ordering direction
 - mostly *fully connected*: every lower with every upper
 - *sparsely* connected – in specific network kinds, e.g., convolutional
- ◆ *Recurrent* architecture: signal transmission path contains cycles



Perceptron

- ◆ A binary classifier that maps numerical inputs to binary outputs. Its main parts are:
 - *input* layer: reads input vectors of features $x = (x_1, \dots, x_n)$
 - *weights* $w = (w_1, \dots, w_n)$: relative contributions of individual features
 - *bias* b : added to weighted contributions of features, $z = w^T x + b$
 - *step function* used as activation function:
$$f(y) = \begin{cases} 1 & |z \geq 0 \\ 0 & |z < 0 \end{cases}$$





Multilayer perceptron

- ◆ Layered network with ≥ 3 layers ordered feedforward
= in accordance with the direction of signals
- ◆ 0th: input layer, 1st–($L-1$)th: hidden layers, L th: output layer, $L \geq 2$

Input
Layer

Hidden Layer

Output
Layer

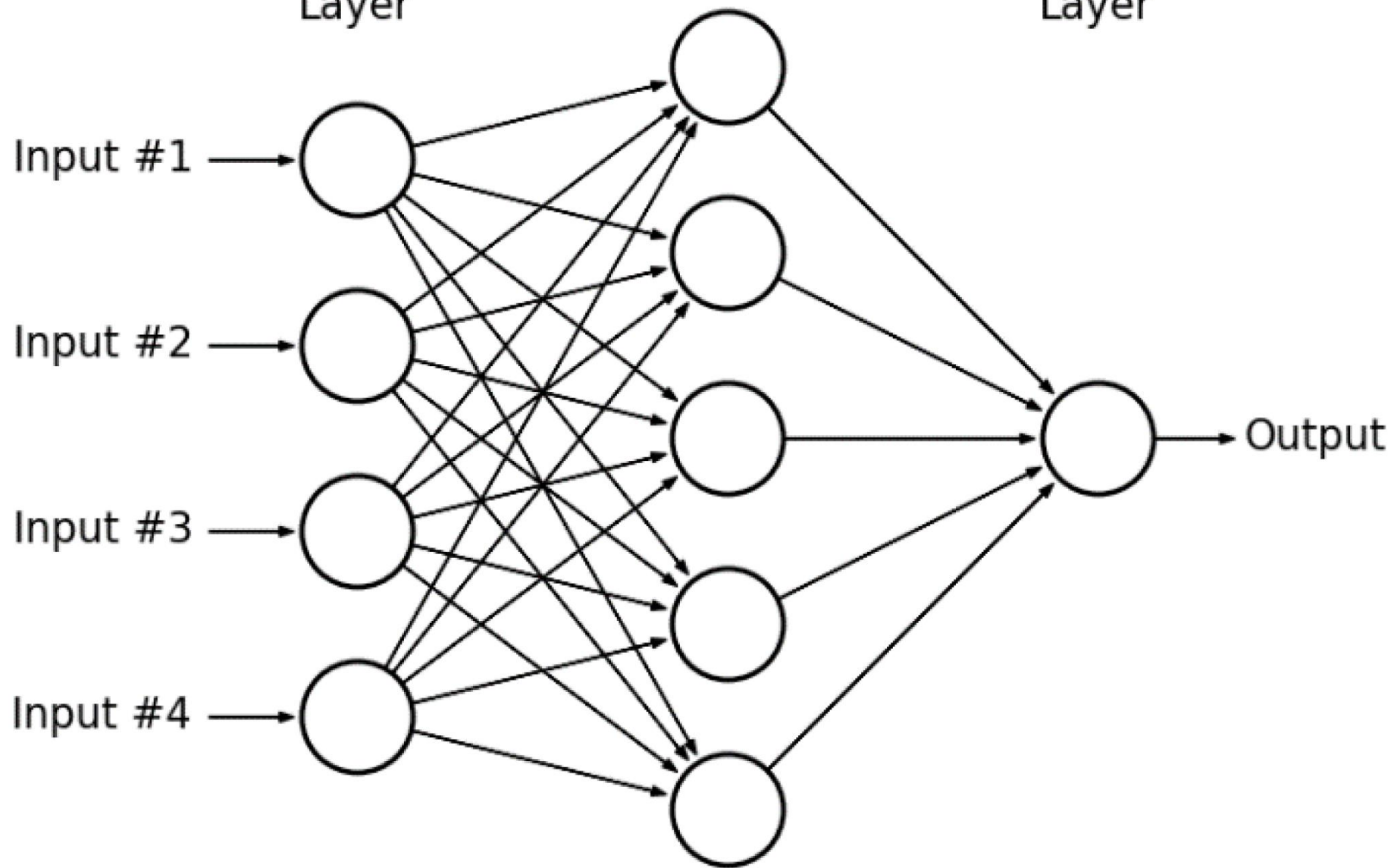
Input #1

Input #2

Input #3

Input #4

Output



Multilayer perceptron

- ◆ Layered network with ≥ 3 layers ordered feedforward
= in accordance with the direction of signals
- ◆ 0th: input layer, 1st–($L-1$)th: hidden layers, L th: output layer, $L \geq 2$
- ◆ E.g., 1 hidden layer $\Rightarrow$ pre-activation activity | final output

$$\begin{array}{l|l} z^{(1)} = W^{(0)}x + b^{(1)} & a^{(1)} = f(z^{(1)}) \\ z^{(2)} = W^{(1)}a^{(1)} + b^{(2)} & \hat{y} = f(z^{(2)}) \end{array}$$

Self-organizing map

- ◆ Mapping with 2|3-dimensional values learned based on similarity
 - $\Rightarrow$ unsupervised, no labels, suitable for clustering and visualization
- ◆ *Input layer: $x \in \mathbb{R}^n$ + map layer $G: m \times m$ neurons grid*
- ◆ Weight $w_i \in \mathbb{R}^n$: represents the pre-image of the neuron $i \in G$
- ◆ Best matching unit for $x \in \mathbb{R}^n$ among neurons from G :

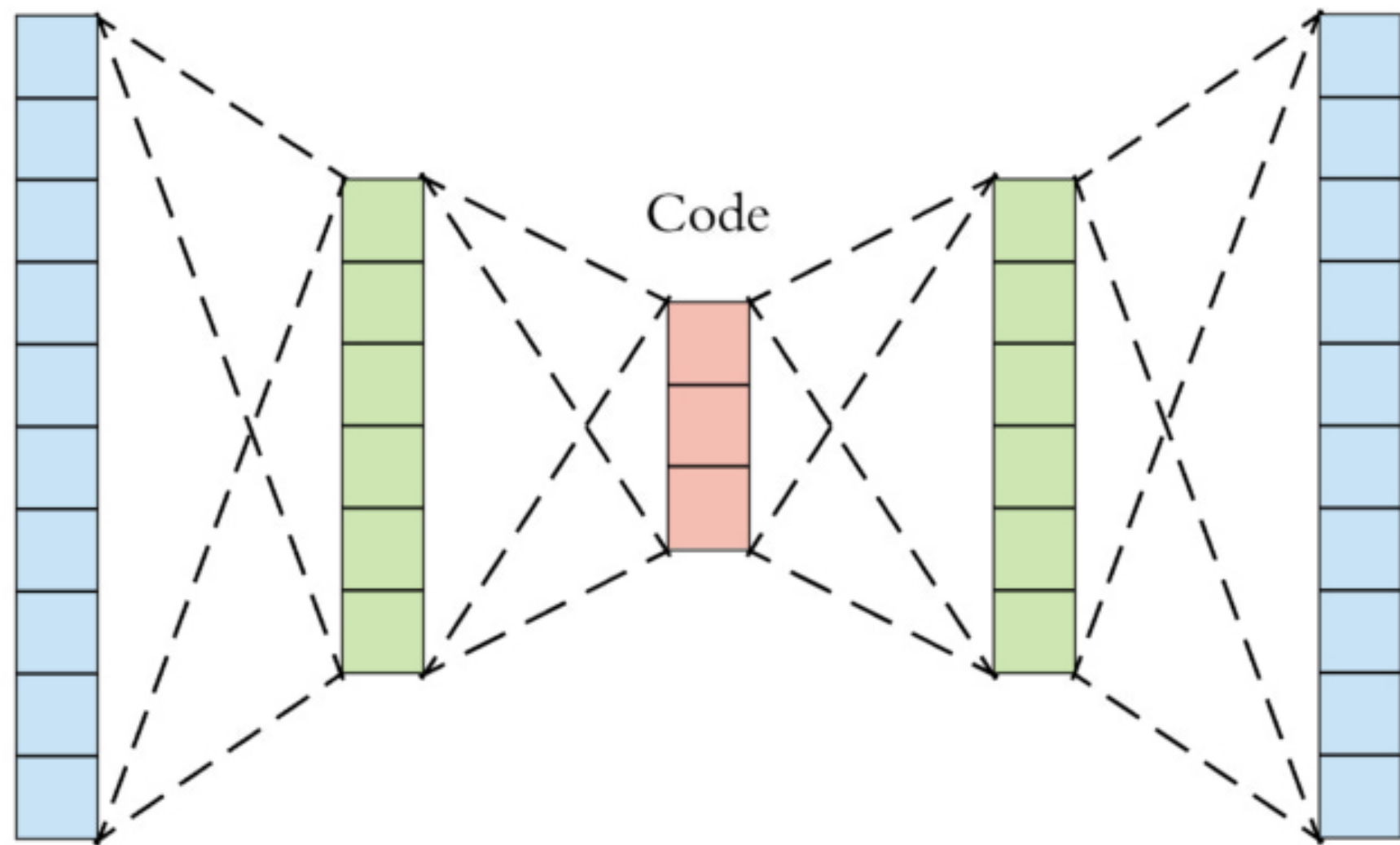
$$i_{\text{BMU}} = \arg \min_{i \in G} \|x - w_i\|$$

Autoencoder

- ◆ *Unsupervised*, learns encoding the input into a lower-dimensional *compressed representations* and then decoding the original
 - purpose: learn an efficient encoding of *essential features*
- ◆ Signal path: input $x \rightarrow$ hidden latent layer $z \rightarrow$ output layer $\hat{x}$
 - possible hidden layers between input – latent, latent – output

Input

Output



Code

Encoder

Decoder

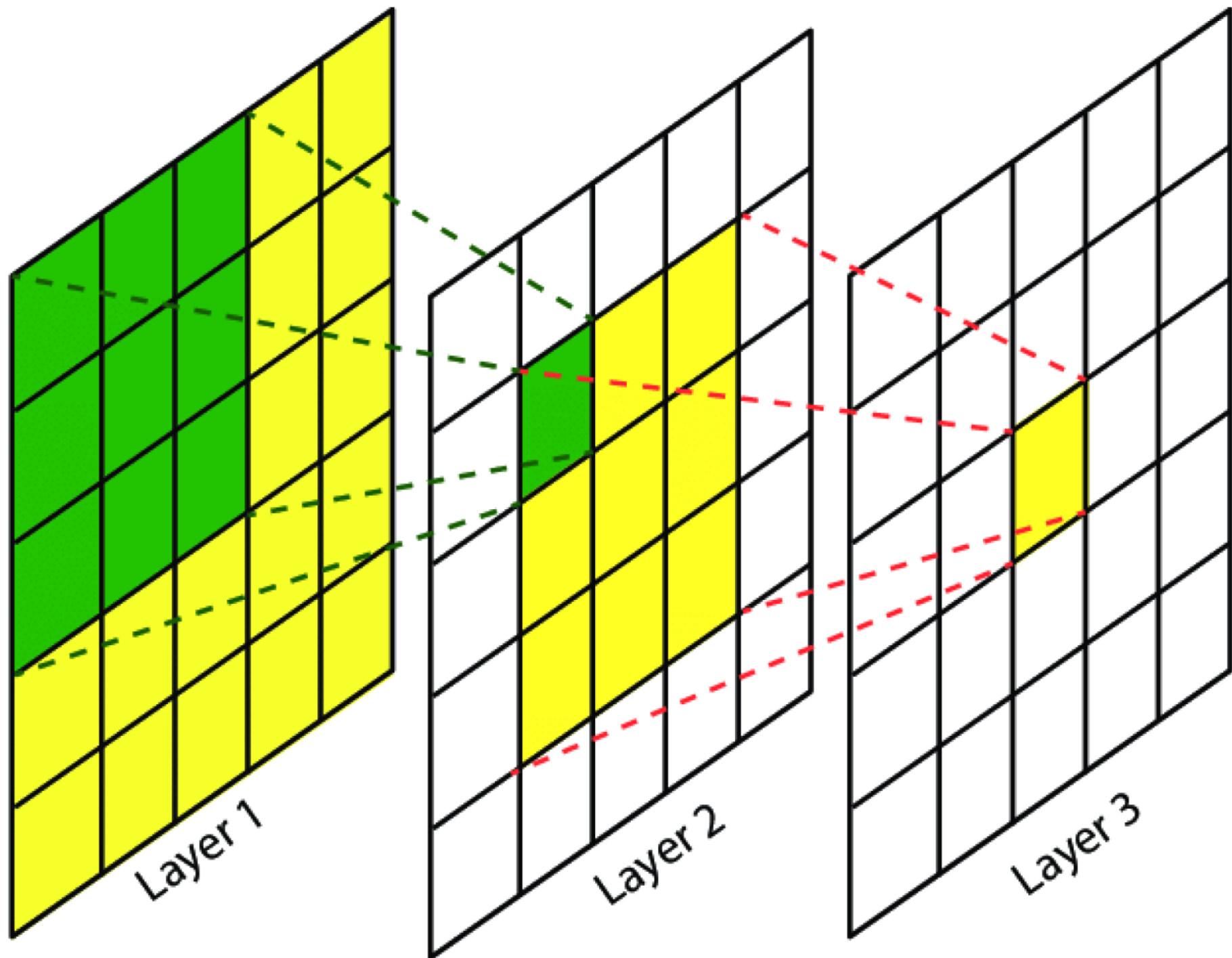
Autoencoder

- ◆ *Unsupervised*, learns encoding the input into a lower-dimensional *compressed representations* and then decoding the original
 - purpose: learn an efficient encoding of *essential features*
- ◆ Signal path: input $x \rightarrow$ hidden latent layer $z \rightarrow$ output layer $\hat{x}$
 - possible hidden layers between input – latent, latent – output

$$\hat{x} = g(z) = \sigma_d(W_d z + b_d) = \sigma_d(W_d f(x) + b_d) = \sigma_d(W_d \sigma_e(W_e x + b_e) + b_d)$$

Convolutional Neural Network (CNN)

- ◆ Exploit the spatial structure of images or other grid data and capture local patterns in them
- ◆ Repeatedly [convolutional + pooling] layers + fully connected layers
- ◆ Convolutional layer: slides filters (aka kernels) over its input, connected only with a local receptive field

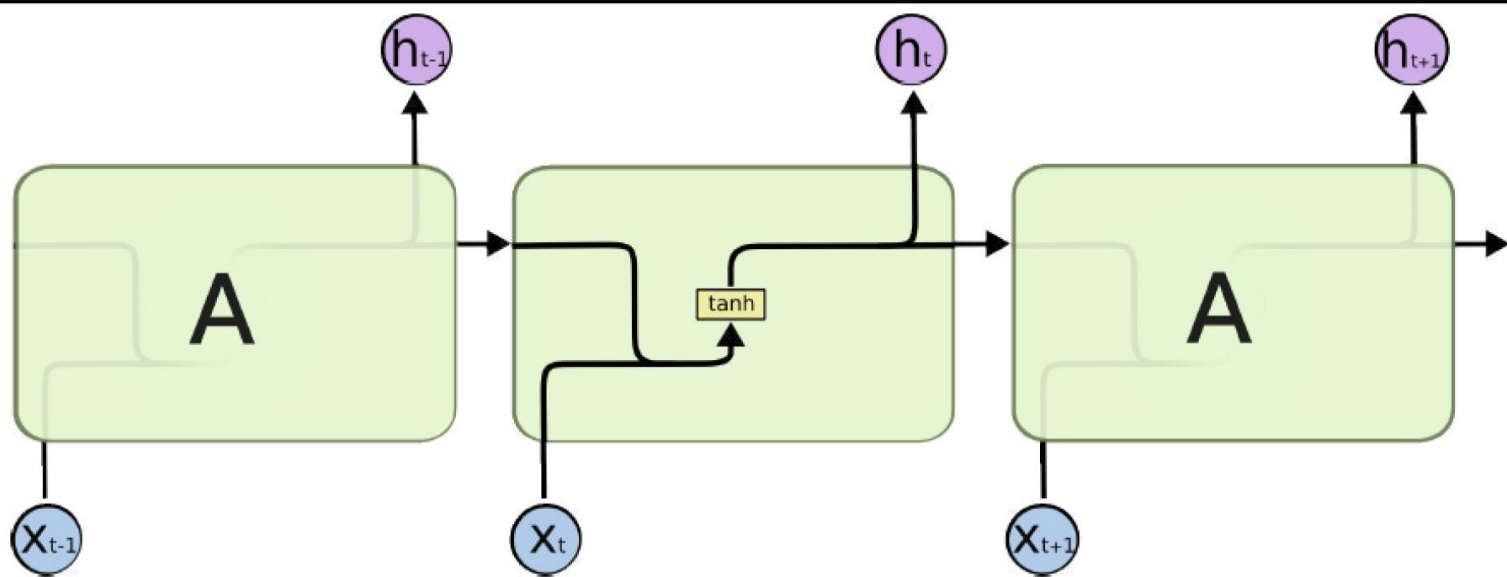


Convolutional Neural Network (CNN)

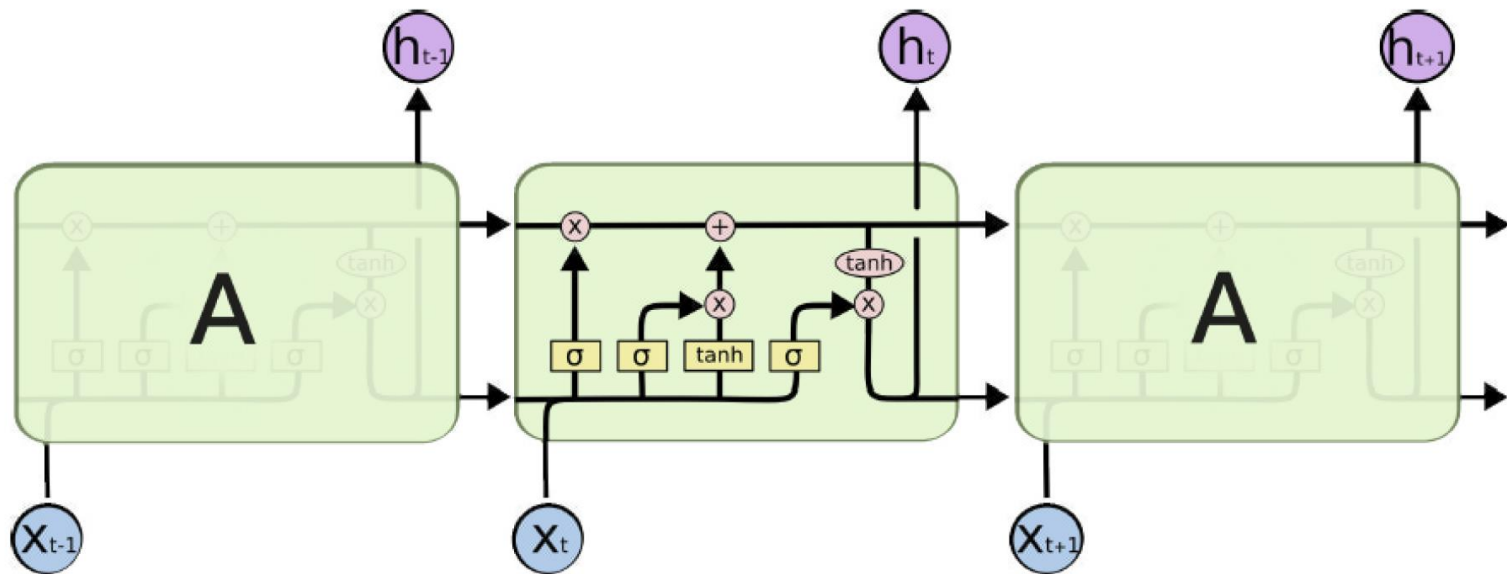
- ◆ Exploit the spatial structure of images or other grid data and capture local patterns in them
- ◆ Repeatedly [convolutional + pooling] layers + fully connected layers
- ◆ Convolutional layer: slides filters (aka kernels) over its input, connected only with a local receptive field
 - *sliding filters output: activation map* (aka feature map)

Recurrent neural networks (RNNs)

- ◆ ANNs considering context or memory of previous inputs
 - $\Rightarrow$ suit for temporal / spacial sequences (text, speech, time-series)
- ◆ Difference to MLPs: loops, e.g., $h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$
- ◆ *Leading* in recent years: *long short-term memory* (LSTM)
 - incorporate a gating mechanism to control information flow
 - $\Rightarrow$ avoid vanishing and exploding gradients of traditional RNNs



Traditional RNN



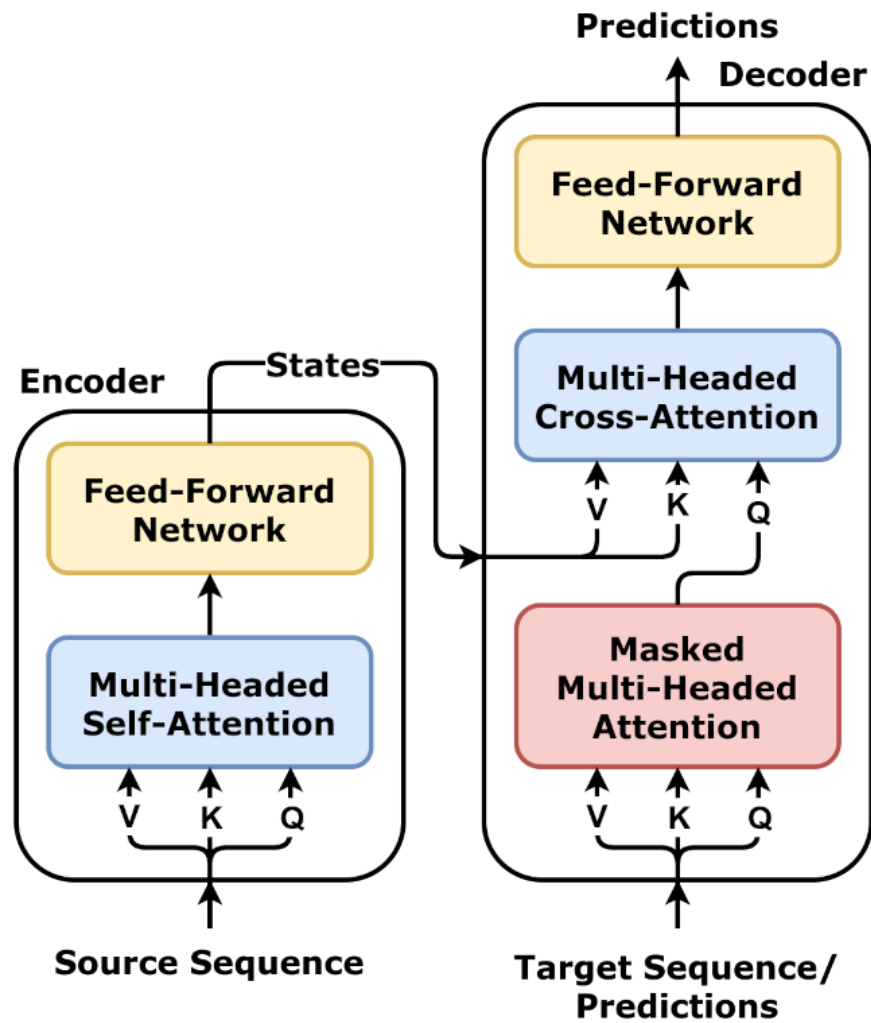
LSTM

Transformer

- ◆ Models sequences via paying attention to new inputs.
- ◆ First, it embeds any sequence $x_1, \dots, x_n$ into $e_1, \dots, e_n \in \mathbb{R}^d$
 - If ordering needed, positional embedding $p_1, \dots, p_n \in \mathbb{R}^d$ is superposed
- ◆ For inputs z_i , the transformer computes *queries* q_i , *keys* k_i , *values* v_i

$$\text{Attention}(z_i) = \sum_{j=1}^n \frac{q_i k_j}{\sum_{j'=1}^n q_i k_{j'}} v_j, q_i = z_i W^Q, k_i = z_i W^K, v_i = z_i W^V$$

$$z_i \in \mathbb{R}^d, q_i, k_i \in \mathbb{R}^{d_k}, v_i \in \mathbb{R}^{d_v}, W^Q, W^K \in \mathbb{R}^{d, d_k}, W^V \in \mathbb{R}^{d, d_v}$$



Graph neural networks (GNNs)

- ◆ Process graphs, represent them as vector space embeddings
 - $\Rightarrow$ applied to social networks, molecular models, knowledge graphs,...
- ◆ At iteration t , embedding follows the message passing paradigm:
 - *message + aggregation functions* process embeddings of node v neighbours:
$$f_{N(v)}^{(t)} = f_{\text{aggregate}}^{(t)} \left(\left\{ f_{\text{message}}^{(t)} \left(h_v^{(t-1)}, h_u^{(t-1)}, e_{vu} \right) \mid u \in \mathcal{N}(v) \right\} \right)$$
 - then, an *update function* applies: $h_v^{(t)} = f_{\text{update}}^{(t)} \left(h_v^{(t-1)}, f_{N(v)}^{(t)} \right)$

General machine learning model

- ◆ *Mathematical representation* of a real-world phenomenon
- ◆ Designed to *generalize* from data used for its learning $\Rightarrow$ able to make predictions for new data
- ◆ *Main steps* in model development: data collection, data preprocessing, feature selection, feature engineering, model training, model evaluation, deployment and monitoring.

Model training

- ◆ Key model development step – *fitting the model + regularization* to data to learn essential patterns in them
- ◆ Data $\supset$ different purpose parts: training + (validation) + testing
 - for choosing from different models / configurations: *cross-validation*
- ◆ Error function, aka loss function: measures model performance
 - examples: MSE in regression, cross-entropy in classification

FORWARD

BACKWARD

HIDDEN
ERROR
↓
UPDATE
WEIGHTS

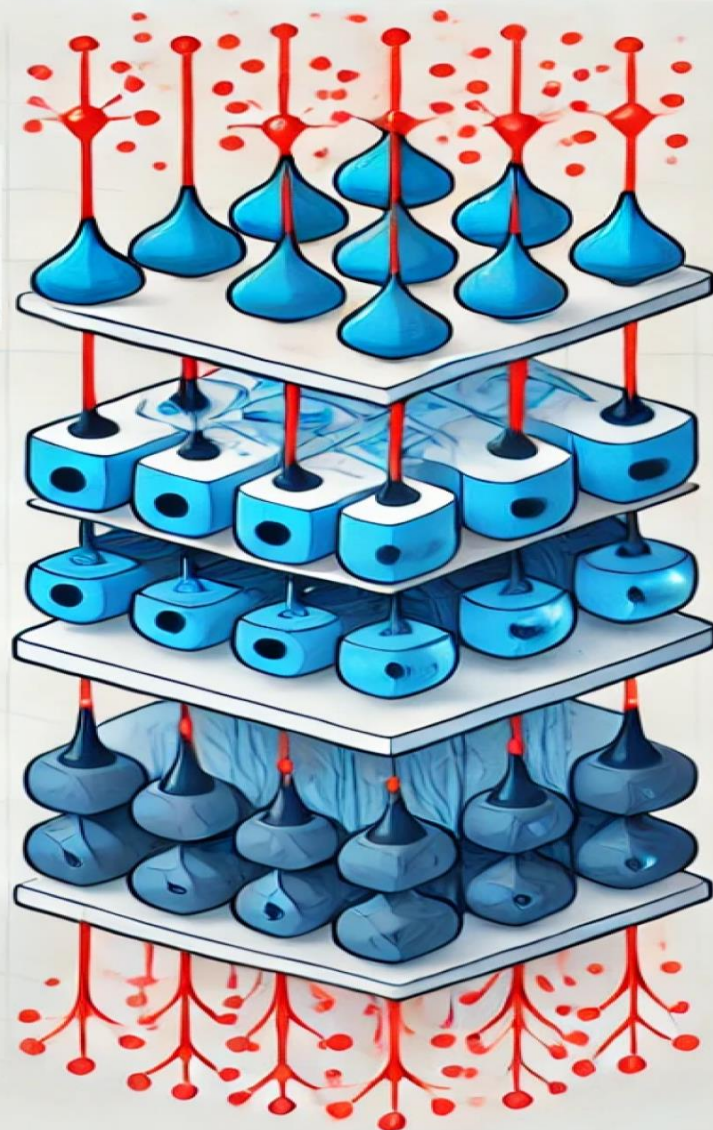


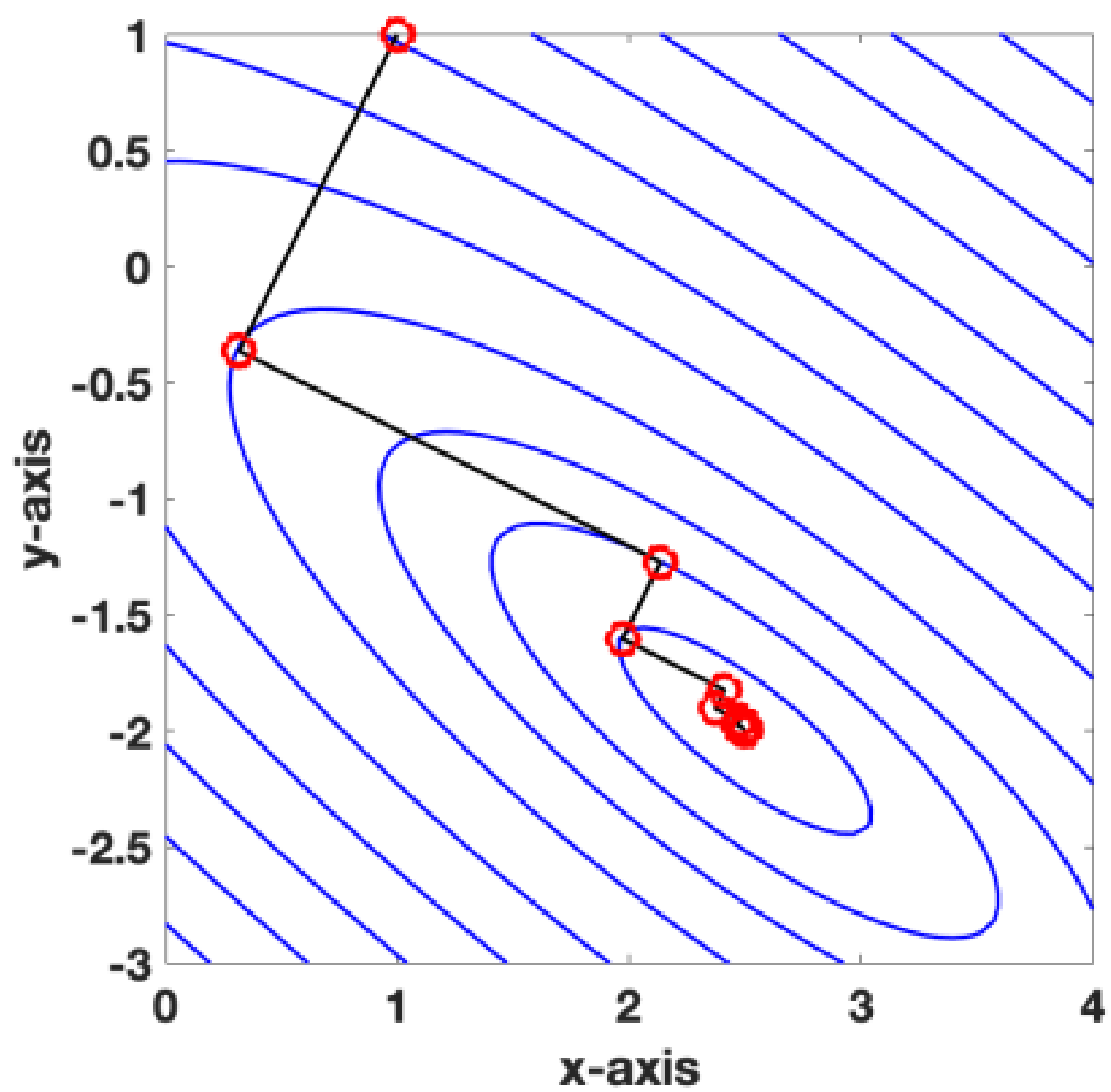
HIDDEN
ERROR
↓
UPDATE
WEIGHTS



UPDATE
WEIGHTS

INPUT
DATA





Feature selection

- ◆ In *high-dimensional* data, $\downarrow$ overfitting $\Rightarrow$ better generalization
 - selection criterion: influence on the model's predictive performance
- ◆ *Filter* methods: rank features without using a model,
based on correlation, mutual information, statistical tests
- ◆ *Wrapper* methods, possibly *embedded* in training: evaluate
feature subsets by training a model on them

Model evaluation

- ◆ Model evaluation: assessing its performance on test data
- ◆ *Evaluation* measures / *performance measures* are task-specific
 - *regression*: mean {squared | absolute} error ({MSE | MAE}), RMSE

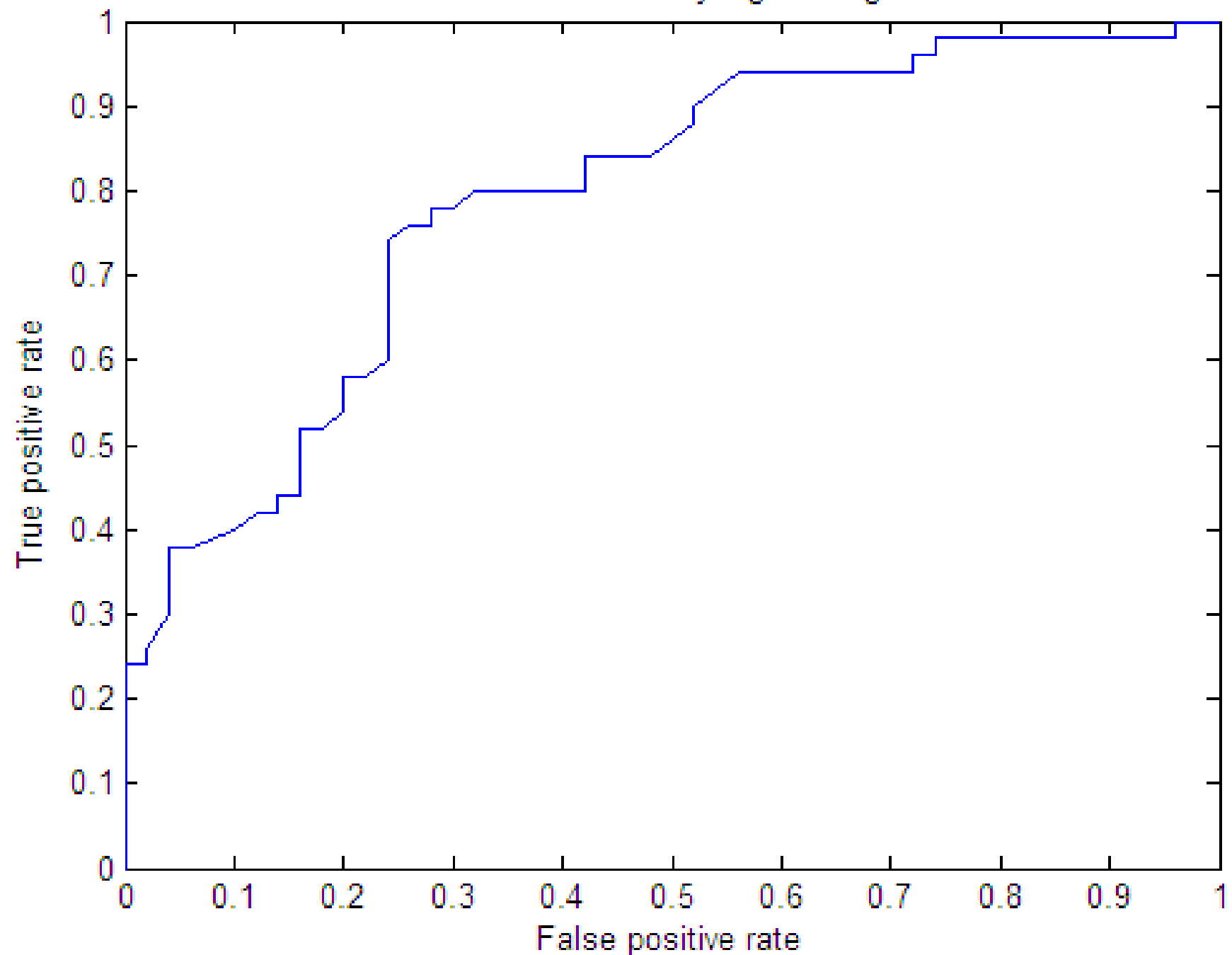
- *classification*: accuracy $\frac{TP+TN}{n}$,

	Classifier	$+(n_{+.})$	$-(n_{-.})$
Correct	$+(n_{+.})$	TP	FN
class	$-(n_{-.})$	FP	TN

precision $\frac{TP}{n_{+.}}$, recall $\frac{TP}{n_{+.}}$,

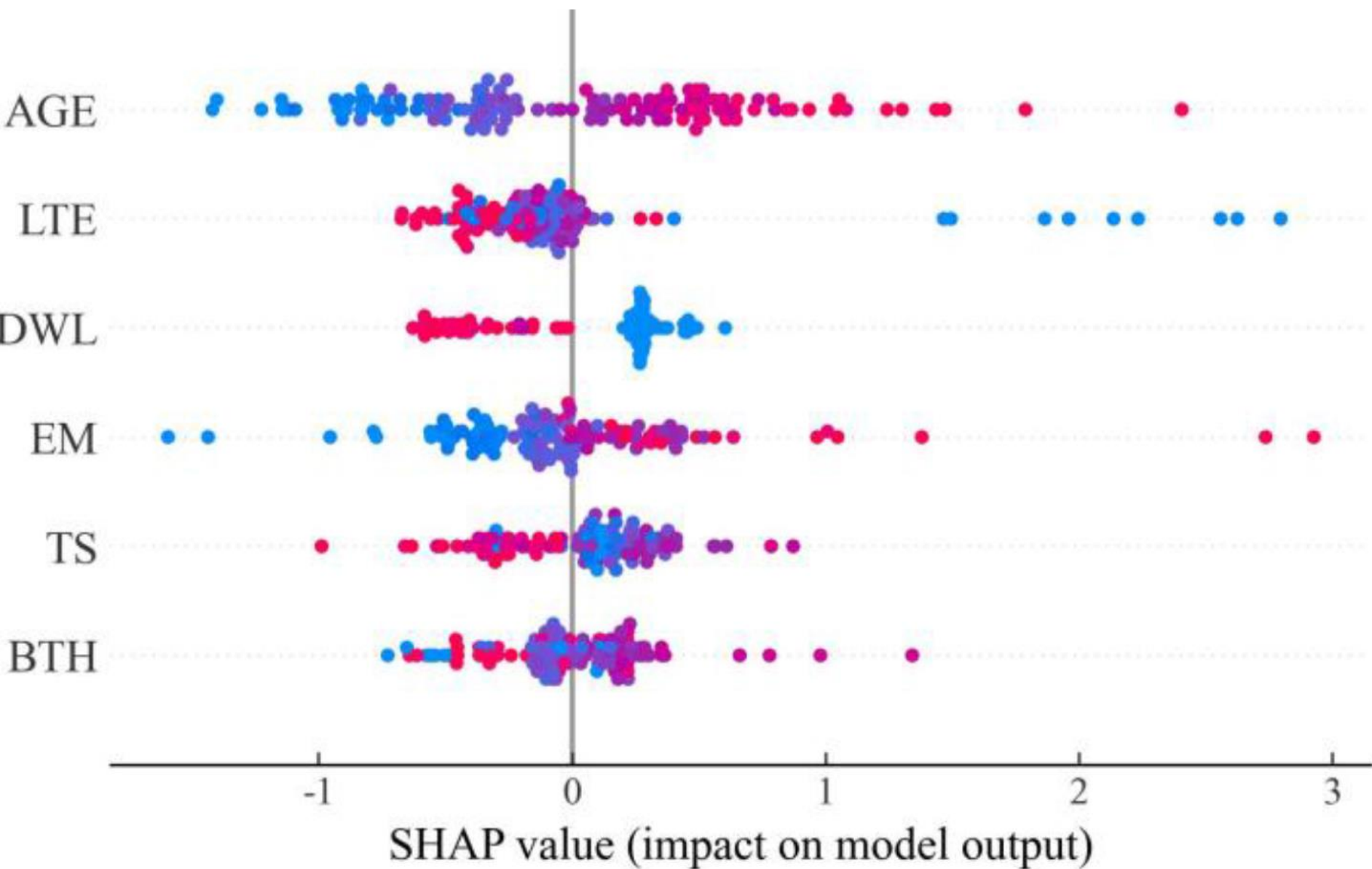
F1-score $\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$, AUC

ROC for classification by logistic regression



Interpretability and explainability

- ◆ Crucial for *understanding* and trust to ML models
- ◆ *Interpretability*: human comprehensibility of model workings
- ◆ *Explainability*: reasoning and justification for model's behavior
 - global: for overall behavior, local: for individual predictions
- ◆ Explanation kind *LIME*: interpretable approximations of predictions
- ◆ Explanation kind *SHAP*: with Shapley values ← cooperative games

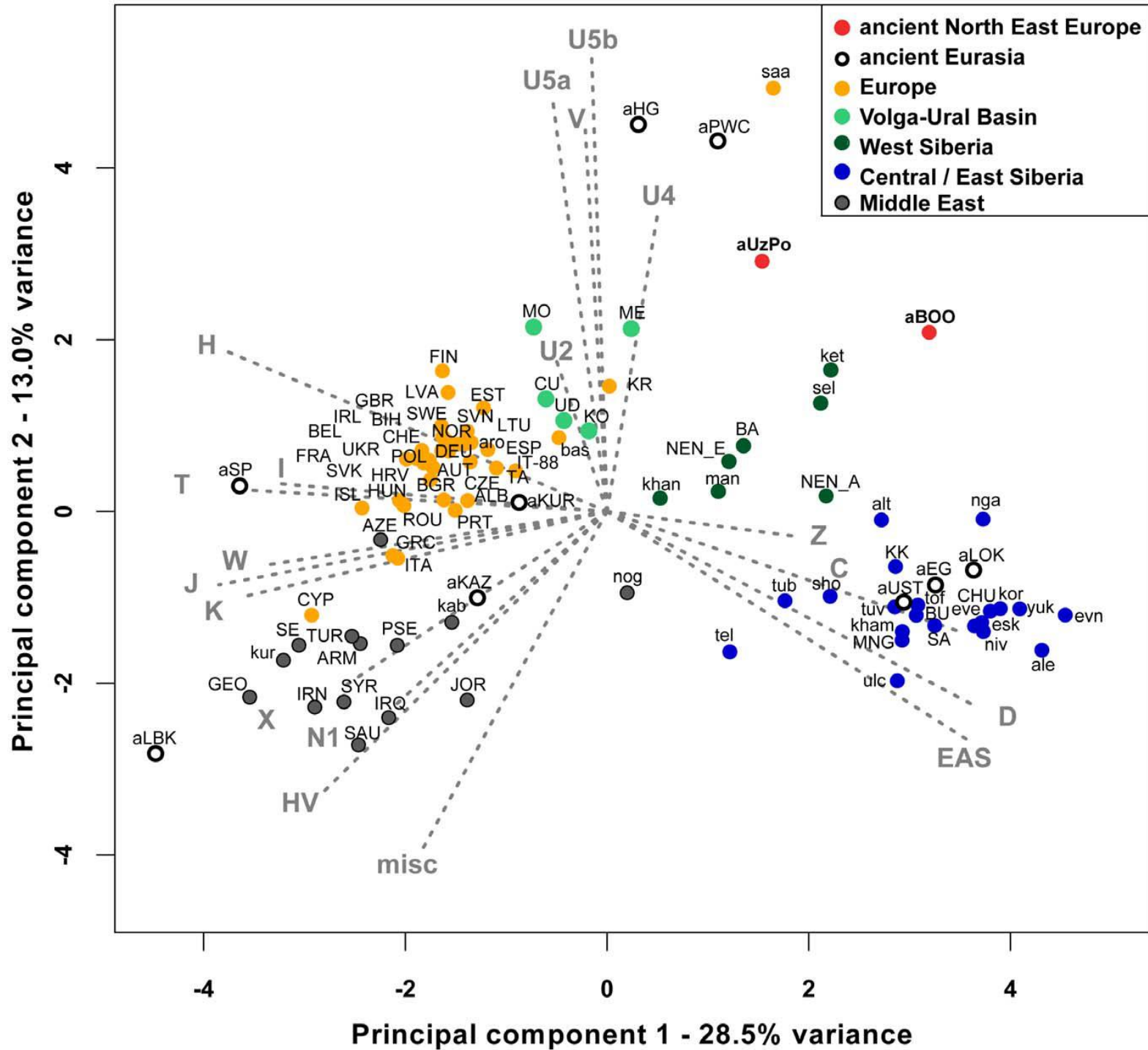


Three basic kinds of learning

1. *Supervised learning*: uses a dataset pairing each input x with a correct output y , learns a mapping f minimizing $|f(x) - y|$
2. *Unsupervised* learning: uses unpaired data, attempts finding there intrinsic patterns (clustering, principal component analysis,...)
3. *Reinforcement* learning: not from static data, but from rewards for actions during *interaction* with the environment

Principal component analysis (PCA)

- ◆ Linear transformation of data into new orthogonal coordinates
 - these *coordinates* are called principal components
 - 1. captures the *maximum variance*, 2. the maximum remaining, etc.



Principal component analysis (PCA)

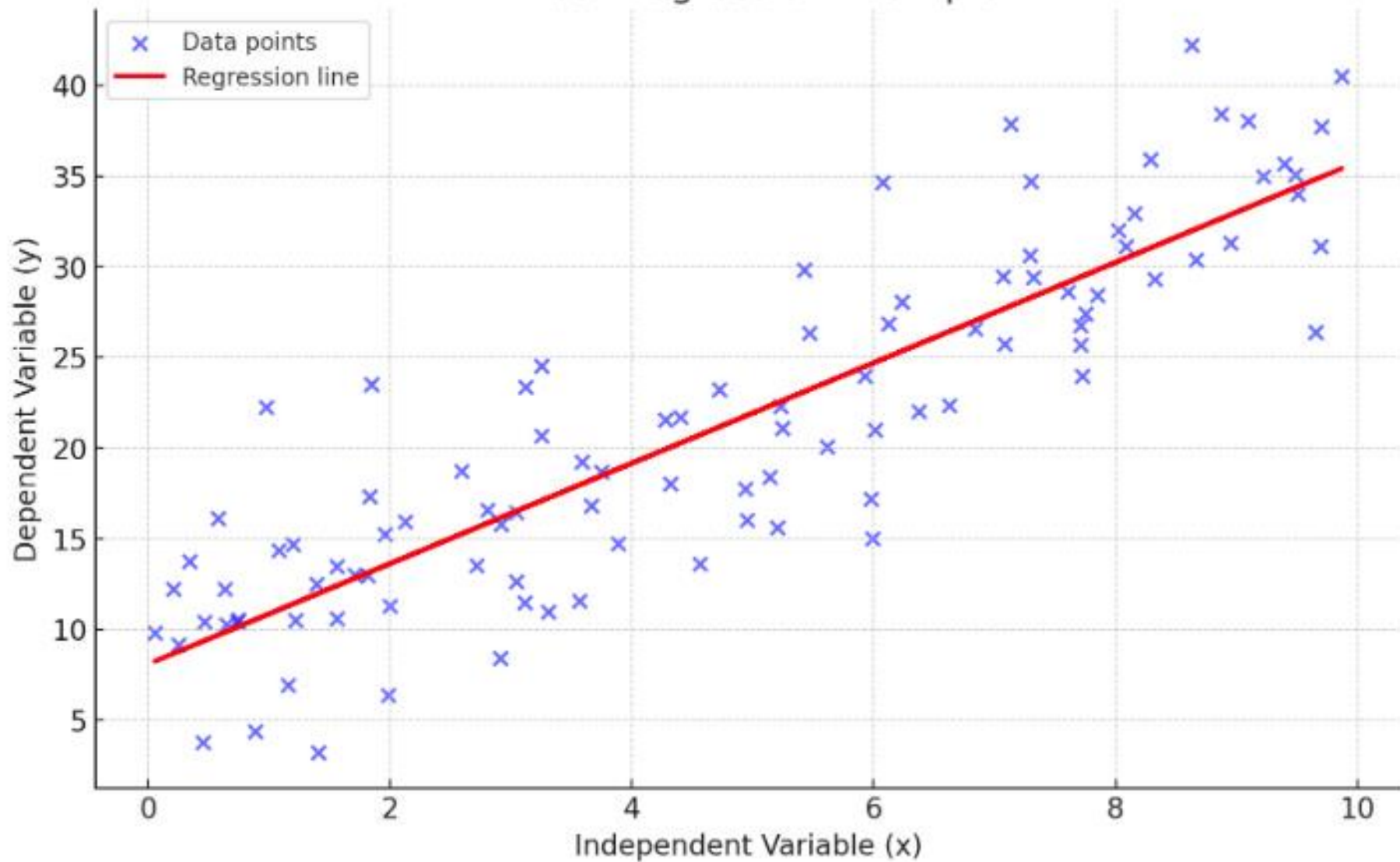
- ◆ Linear transformation of data into new orthogonal coordinates
 - these *coordinates* are called principal components
 - 1. captures the *maximum variance*, 2. the maximum remaining, etc.
- ◆ For n observations, p features: $X \in \mathbb{R}^{n,p}$ standardized to $\tilde{X}$, $\tilde{X}_{i,j} = \frac{X_{i,j} - \mathbb{E}[X_{.j}]}{\sqrt{\text{Var}X_{.j}}}$
- ◆ $\tilde{X}$ principal components Z are projections onto $\text{Var}X$ eigenvectors Q :

$$Z = \tilde{X}Q, \text{Var}X = Q\text{diag}(\lambda_1, \dots, \lambda_p)Q^\top$$

Linear regression

- ◆ Describes a linear relationship $Y = X\beta + \varepsilon$ between a dependent variable $Y \in \mathbb{R}^n$ and independent predictors $X \in \mathbb{R}^{n,d}$ with $\varepsilon \sim N(0, \sigma^2)$

Linear Regression Example



Linear regression

- ◆ Describes a linear relationship $Y = X\beta + \varepsilon$ between a dependent variable $Y \in \mathbb{R}^n$ and independent predictors $X \in \mathbb{R}^{n,d}$ with $\varepsilon \sim N(0, \sigma^2)$
- ◆ β is estimated using the *ordinary least squares* method:

$$\hat{\beta} = \arg \min_{\beta'} \text{RSS} = \arg \min_{\beta'} \|Y - X\beta\|^2 = (X^\top X)^{-1} X^\top Y$$

- ◆ Regression quality is assessed with $R^2 = \frac{\text{RSS}}{\text{TSS}} = \frac{\|Y - X\beta\|^2}{\sum_{i=1}^n \left(Y_i - \frac{1}{n} \sum_{j=1}^n Y_j\right)^2}$
 - *RSS* | *TSS*: residual | total sum of squares

Logistic regression

- ◆ Models positive class probability in binary classification:

$$P(y = 1|x_1, \dots, x_n) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- z aggregates the inputs $x_1, \dots, x_n$ linearly: $z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$

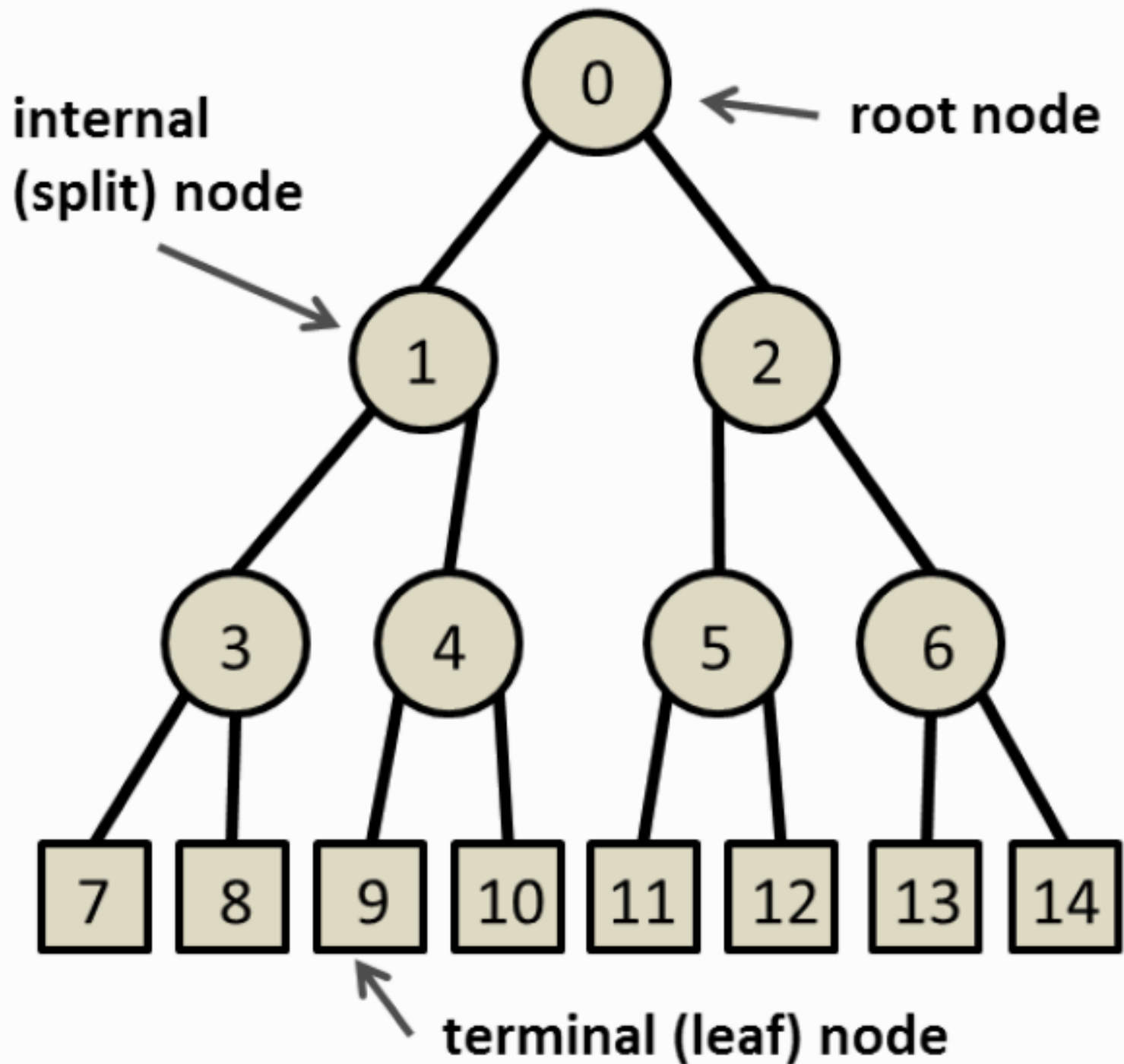
- ◆ $z = \log \frac{P(y = 1|x_1, \dots, x_n)}{P(y = 0|x_1, \dots, x_n)}$ is called *logit* or log-odds

- ◆ Estimation of $\beta_0, \dots, \beta_n$ with maximum likelihood:

$$\text{likelihood } L = \prod_{i=1}^n P(x_i)^{y_i} (1 - P(x_i))^{(1-y_i)}$$

Classification and regression trees

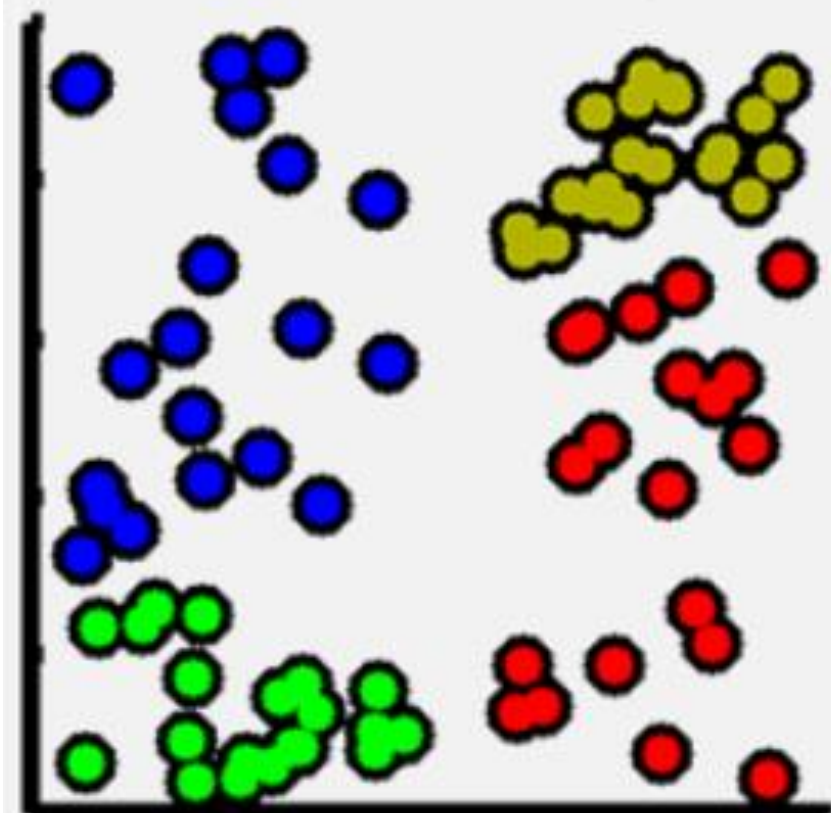
- ◆ Binary trees, with nodes testing Boolean conditions, their validity determine the branch where the input continues



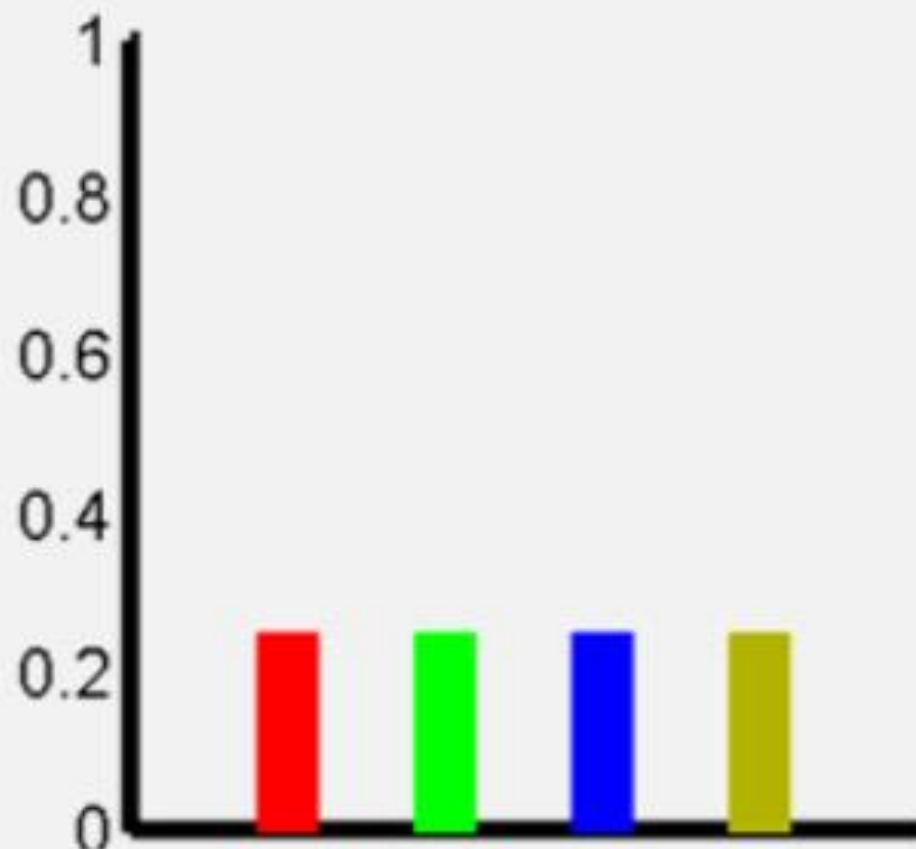
Classification and regression trees

- ◆ Binary trees, with nodes testing Boolean conditions, their validity determine the branch where the input continues
- ◆ Classification conditions minimize *gain* (=entropy) or *Gini impurity*

data before split



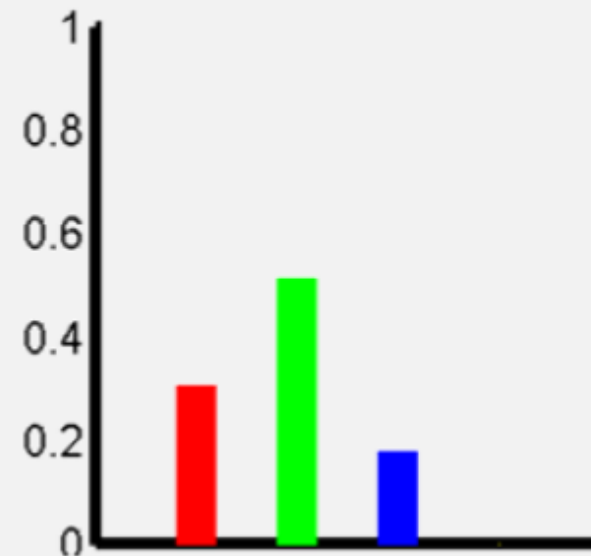
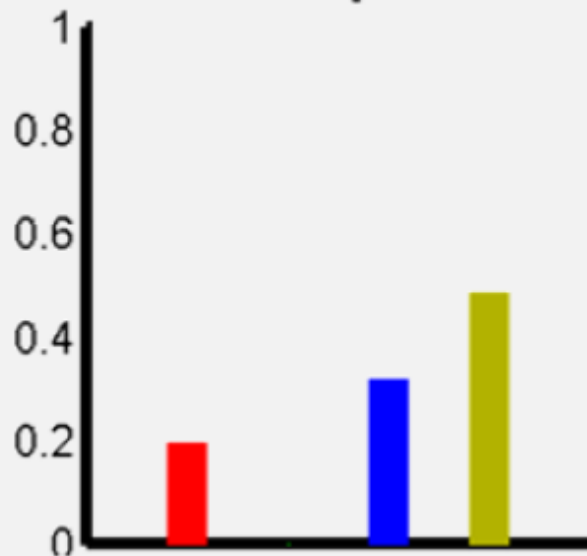
class distribution



Info Gain = 0.40

top

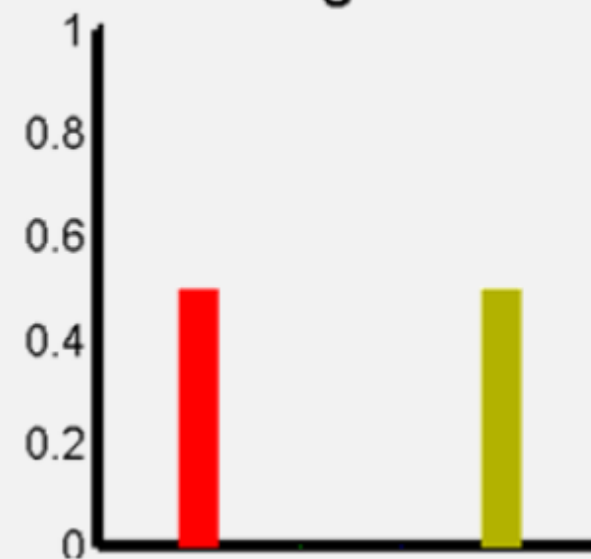
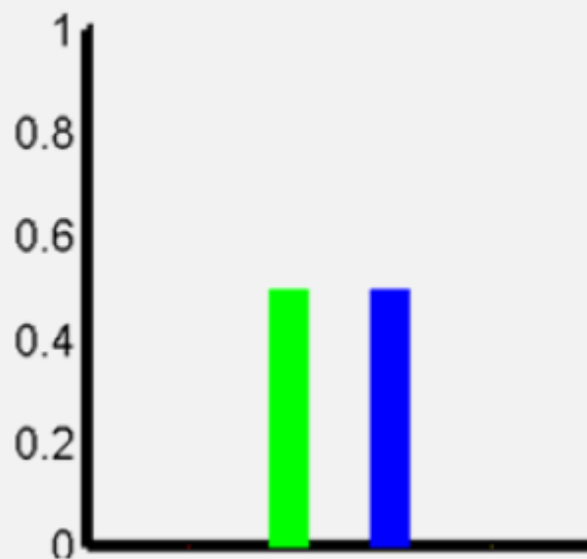
bottom

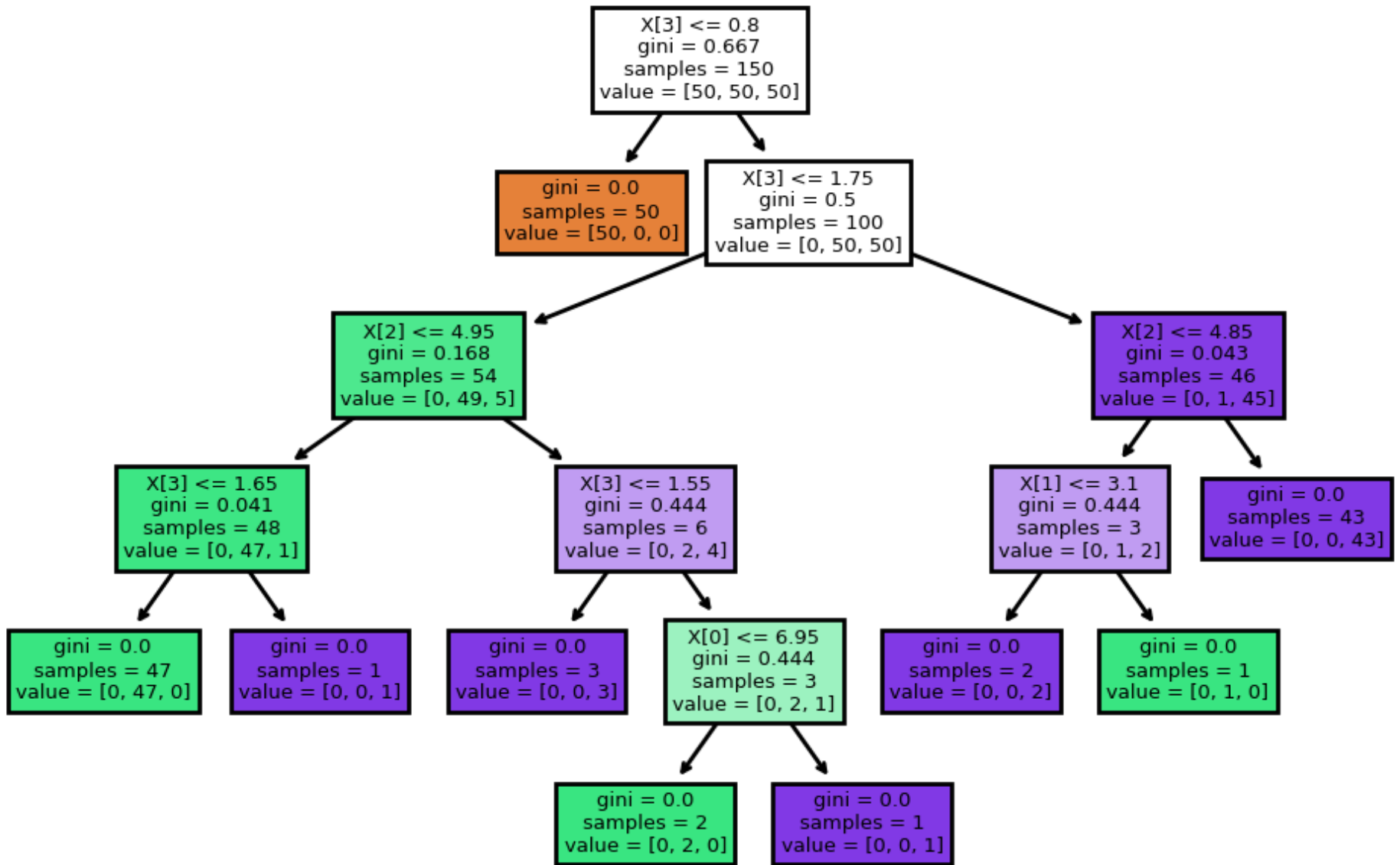


Info Gain = 0.69

left

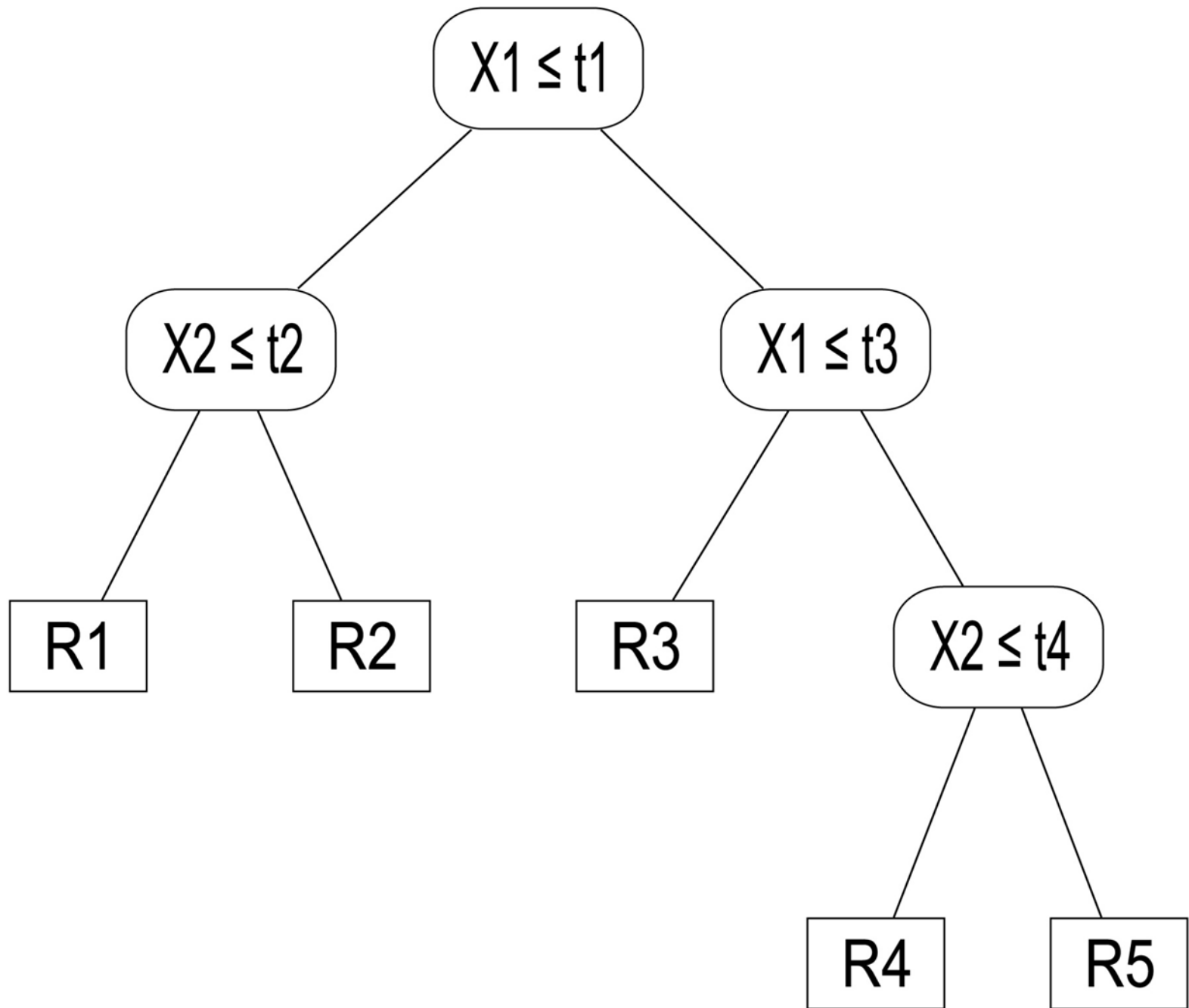
right

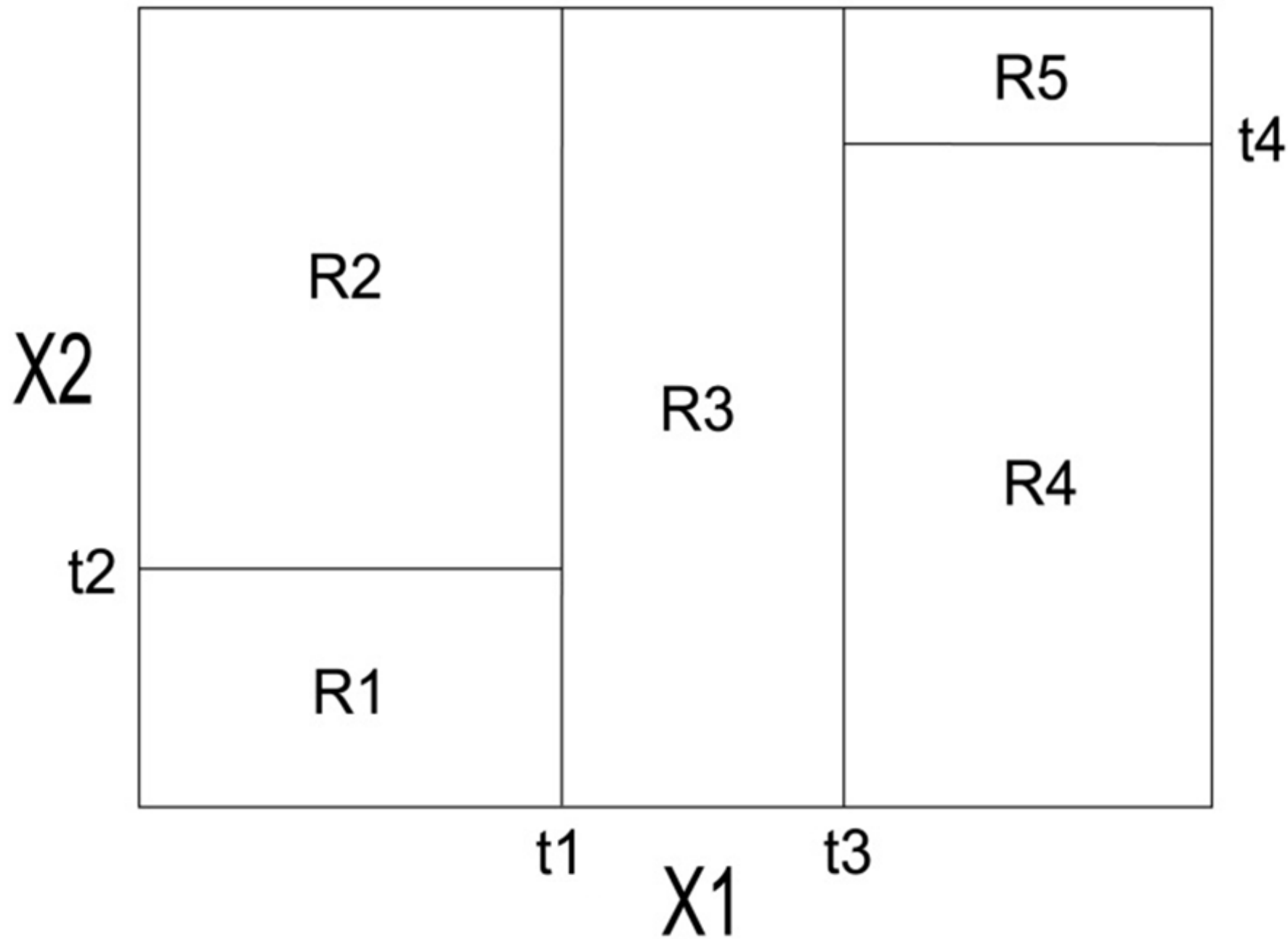




Classification and regression trees

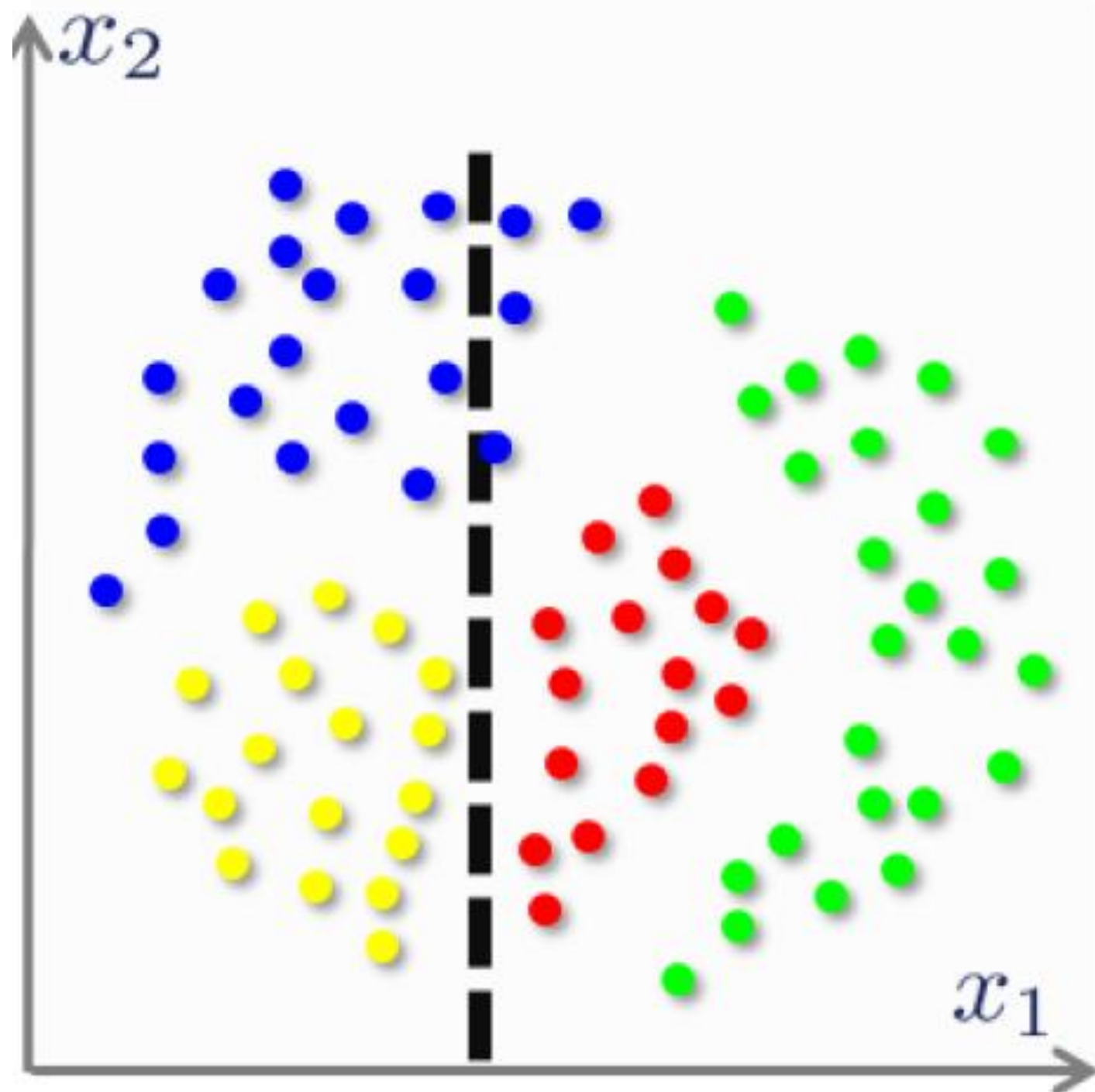
- ◆ Binary trees, with nodes testing Boolean conditions, their validity determine the branch where the input continues
- ◆ Classification conditions minimize *gain* (=entropy) or *Gini impurity*
- ◆ Regression conditions minimize *variance* within the subsets
- ◆ Basic kind: conditions $\sim$ axis-perpendicular hyperplanes

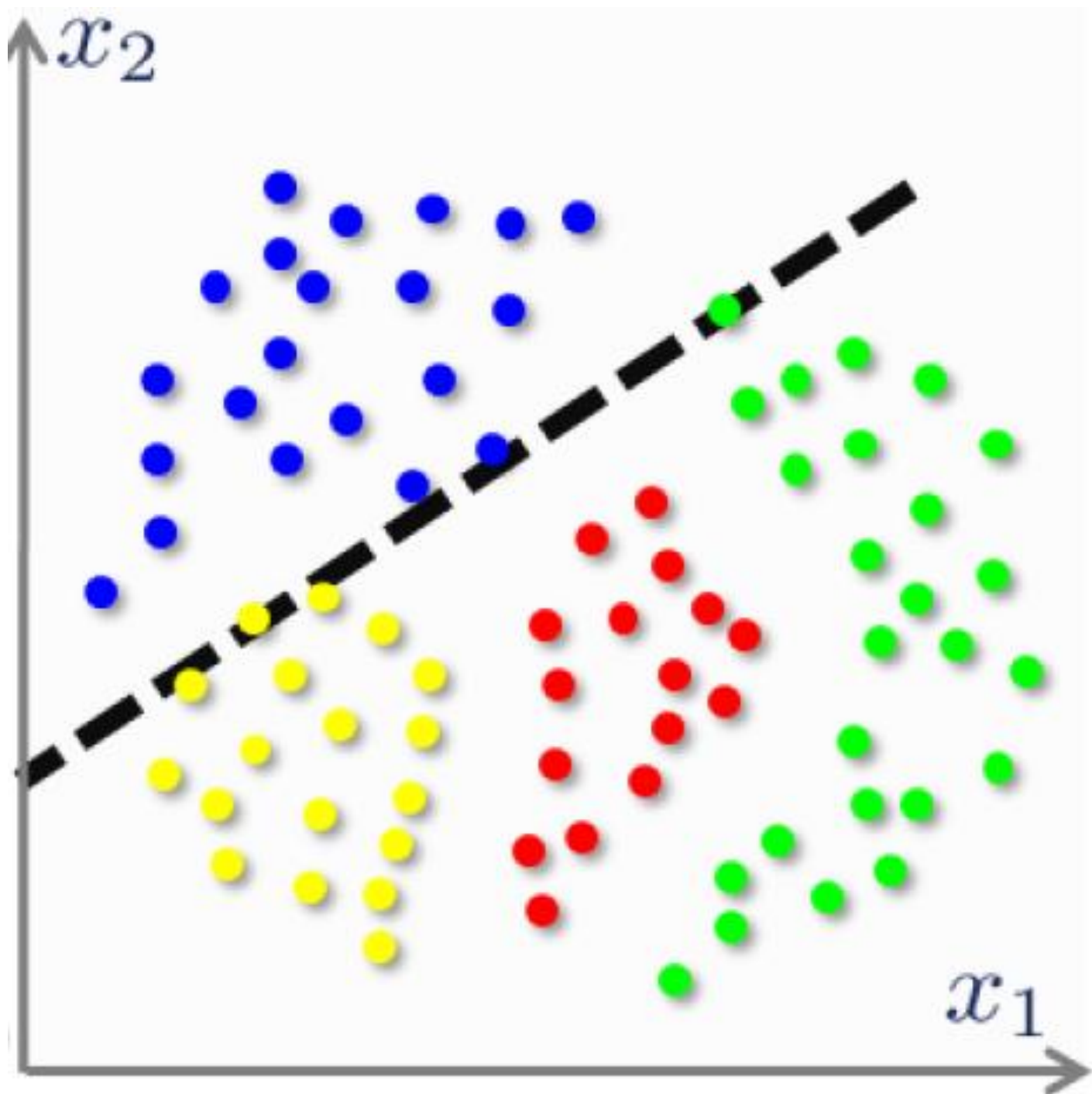


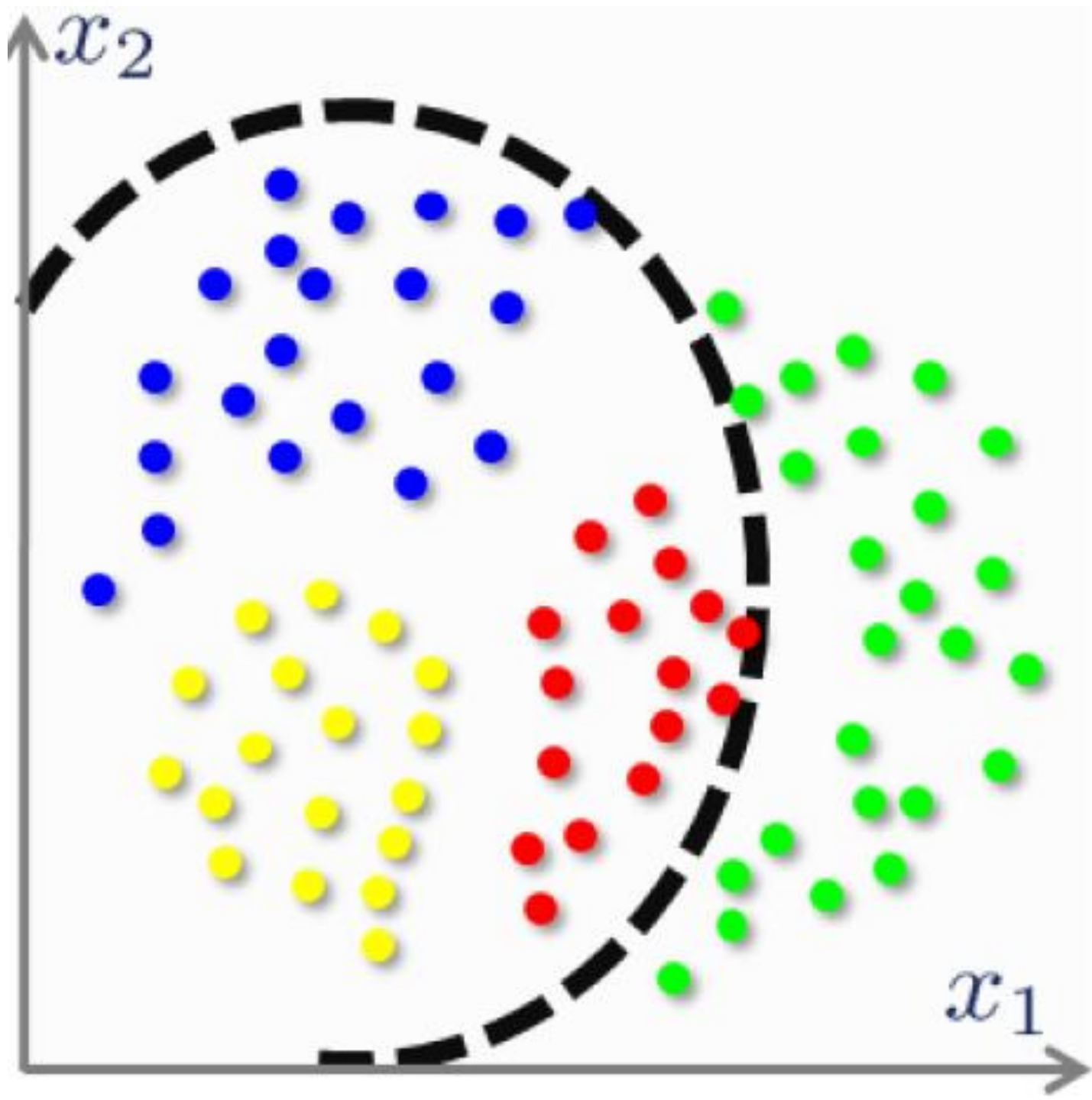


Classification and regression trees

- ◆ Binary trees, with nodes testing Boolean conditions, their validity determine the branch where the input continues
- ◆ Classification conditions minimize *gain* (=entropy) or *Gini impurity*
- ◆ Regression conditions minimize *variance* within the subsets
- ◆ Basic kind: conditions $\sim$ axis-perpendicular hyperplanes
 - alternative kinds: $\sim$ general hyperplanes, $\sim$ quadratic surfaces





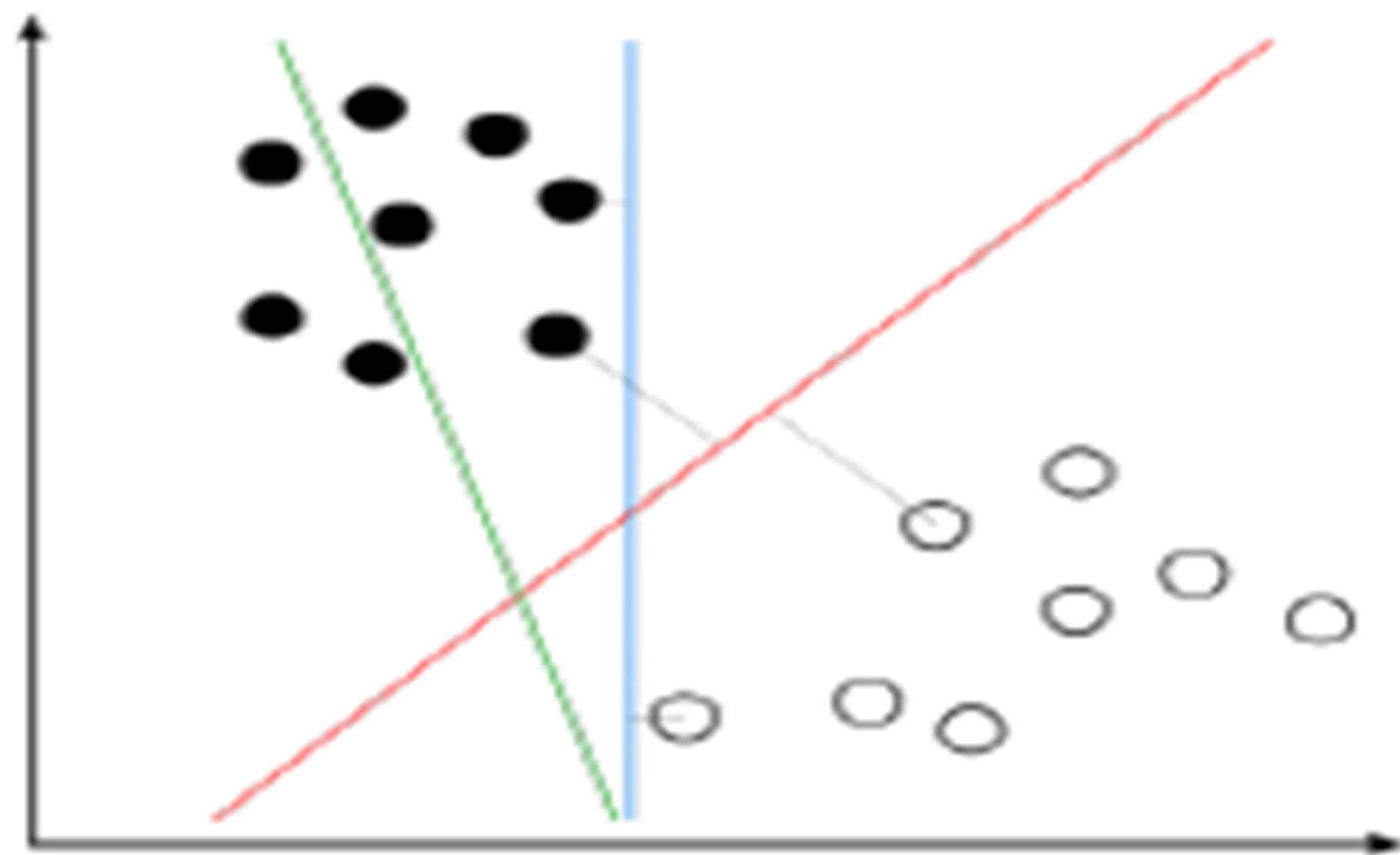


Support vector machine (SVM)

- ◆ Simplest: *2 classes $C_{\pm}$ linearly separable*, classifier $F: \mathbb{R}^d \rightarrow \{1, -1\}$,

$F(x) = 1$ iff $H: w^{\top}x + b \geq 0$ with q training pairs, $\|w\| \leq \Lambda > 0 \Rightarrow$:

$$\exists \gamma > 0: P\left((C_+ \cap F^{-1}(-1)) \cup (C_- \cap F^{-1}(1))\right) < \sqrt{\frac{\gamma}{q} \left(\frac{\Lambda^2 \left(\max_{i=1..q} \|x_i\| \right)^2}{\left(\min_{i=1..q} d(H, x_i) \right)^2} \ln^2 q + |\ln P(x \neq 0)| \right)}$$



Support vector machine (SVM)

- ◆ Simplest: *2 classes $C_{\pm}$ linearly separable*, classifier $F: \mathbb{R}^d \rightarrow \{1, -1\}$,

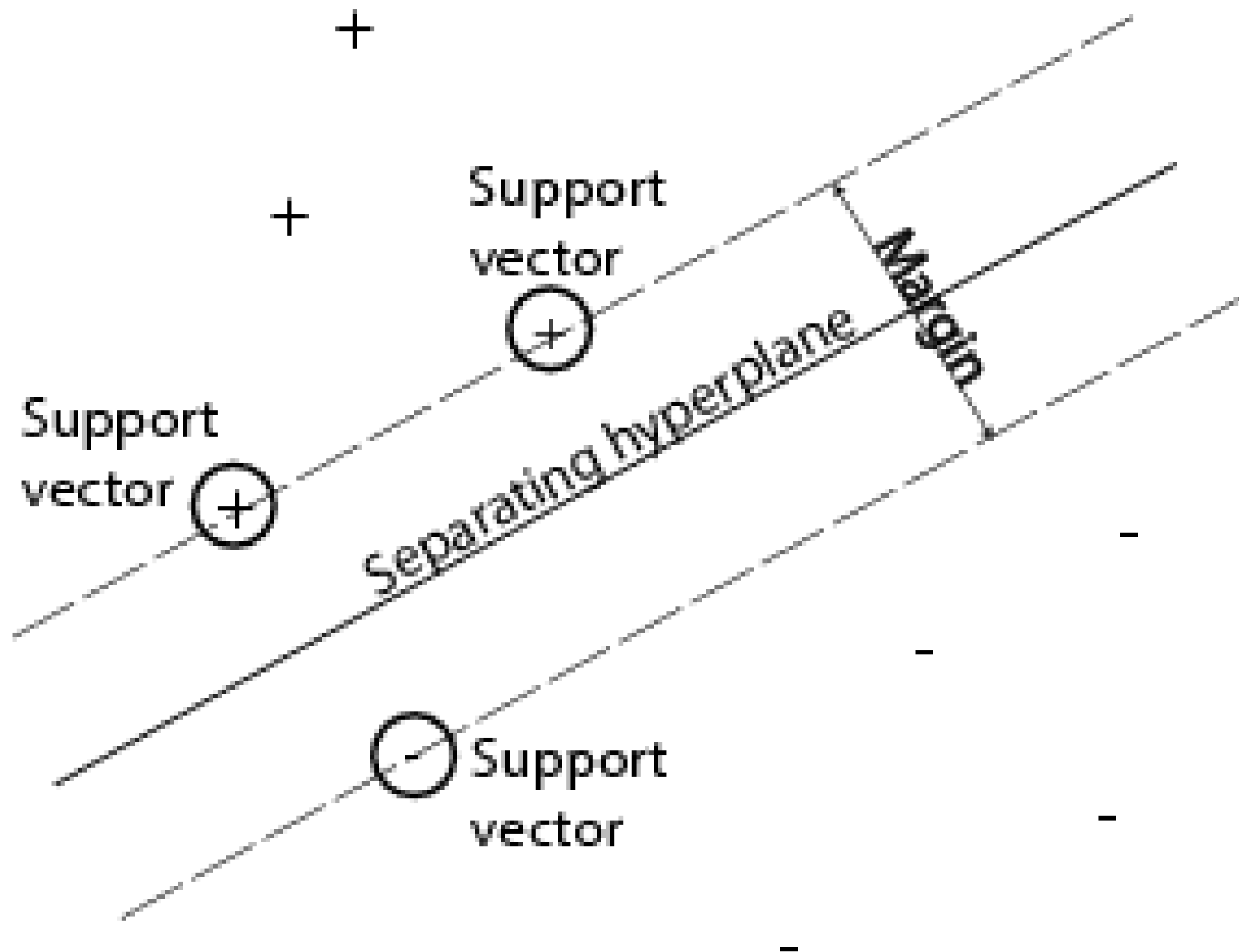
$F(x) = 1$ iff $H: w^T x + b \geq 0$ with q training pairs, $\|w\| \leq \Lambda > 0 \Rightarrow$:

$$\exists \gamma > 0: P\left((C_+ \cap F^{-1}(-1)) \cup (C_- \cap F^{-1}(1))\right) < \sqrt{\frac{\gamma}{q} \left(\frac{\Lambda^2 \left(\max_{i=1..q} \|x_i\| \right)^2}{\left(\min_{i=1..q} d(H, x_i) \right)^2} \ln^2 q + |\ln P(x \neq 0)| \right)}$$

- ◆ $\Rightarrow$ optimize $\min \|w\|^2$ and $\max \delta$ constrained by: $d(H, x_i) \geq \frac{\delta}{\|w\|}, i = 1 \dots q$

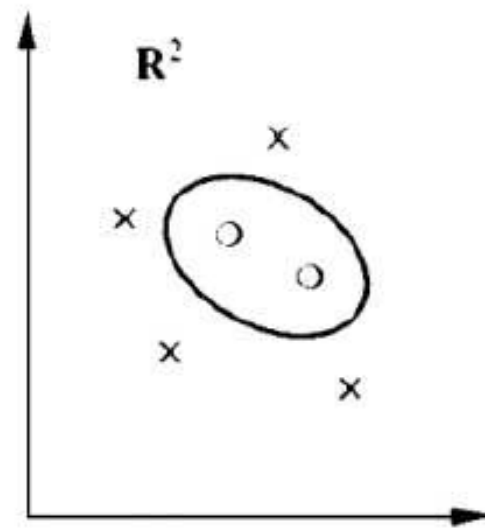
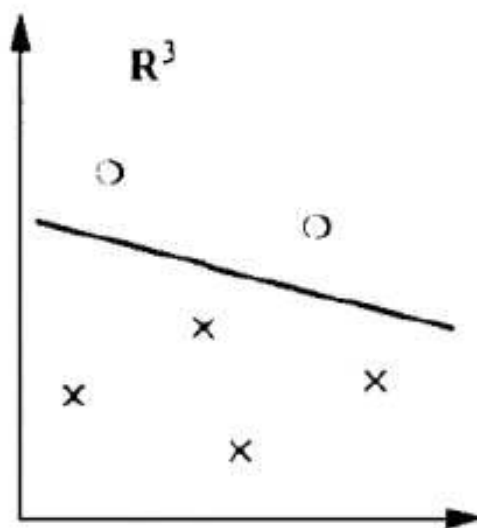
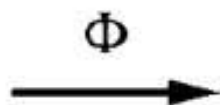
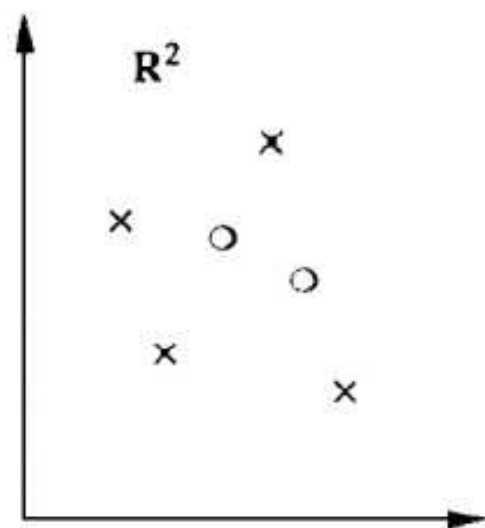
- ◆ Optimal $\hat{w}, \hat{\delta}$ depend only on x_i with $d(H, x_i) = \frac{\delta}{\|w\|} \Rightarrow x_i \in \text{support h-planes}$

$$\hat{w} = \sum_{i \in S} \hat{\alpha}_i c_i x_i, (\hat{\alpha}_1, \dots, \hat{\alpha}_q, \hat{\delta}) = \arg \max_{\alpha_1, \dots, \alpha_q} 2\delta \sum_{i=1}^q \alpha_i - \sum_{i,j=1}^q \alpha_i \alpha_j c_i c_j x_i^T x_j, S = \left\{ i: d(H, x_i) = \frac{\delta}{\|w\|} \right\}$$



SVM for linearly nonseparable classes

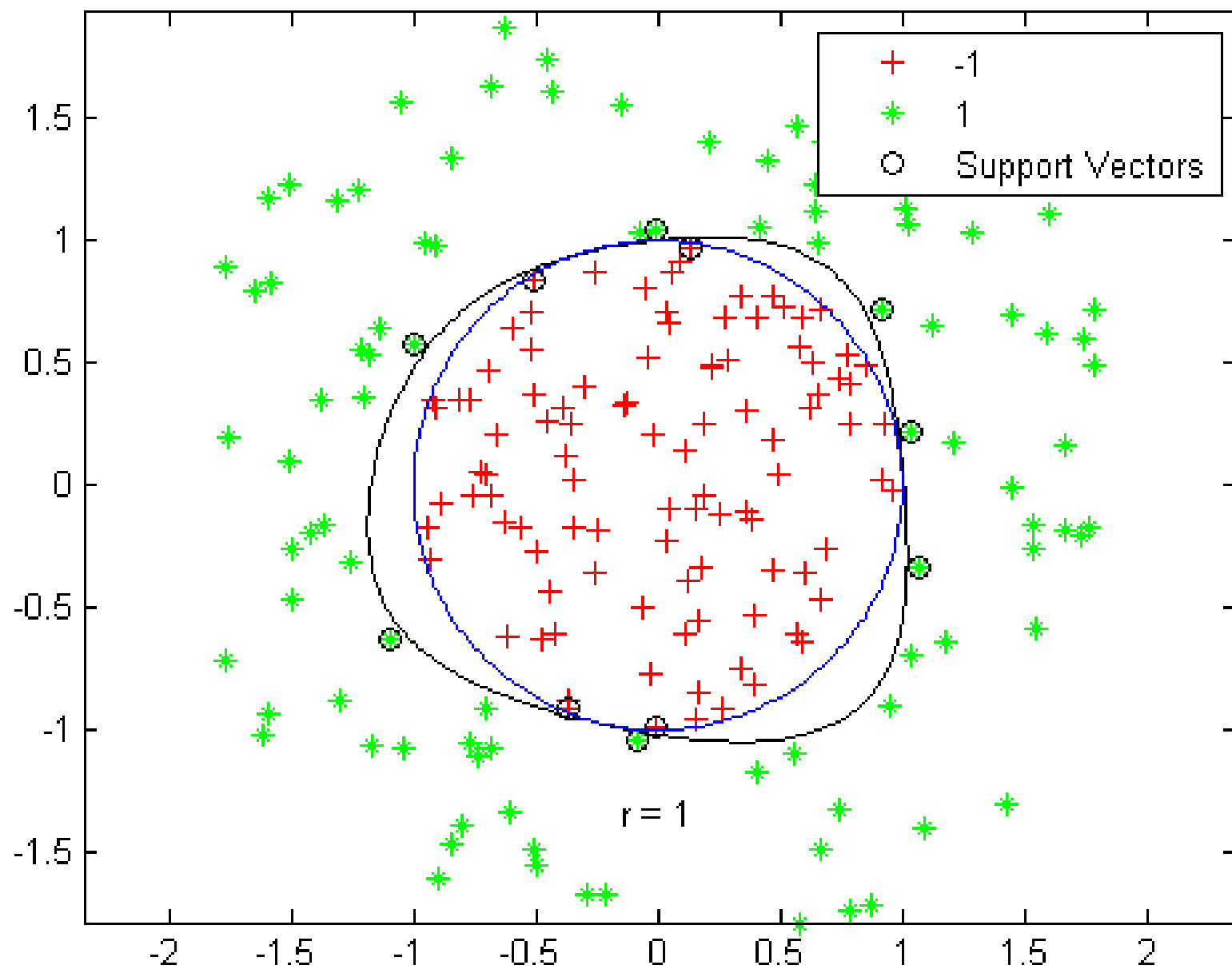
- ◆ *Transformation* into a *higher dimension*



SVM for linearly nonseparable classes

- ◆ *Transformation* into a *higher dimension*: A general-position $S \subset \mathbb{R}^d$ with $d \geq \#M - 1$ has any $C \subset S, C \neq \emptyset, S$ linearly separable from $S \setminus C$
- ◆ Using a kernel $\kappa: X^2 \rightarrow \mathbb{R}$ fulfilling $\kappa(x, y) = \kappa(y, x)$ and for any $x_1, \dots, x_q \in X$ is $\begin{pmatrix} \kappa(x_1, x_1) & \cdots & \kappa(x_1, x_q) \\ \vdots & \ddots & \vdots \\ \kappa(x_q, x_1) & \cdots & \kappa(x_q, x_q) \end{pmatrix}$ positive semidefinite
 - $\#M$ -dimensional vector space: $V = \{\sum_{i=1}^q a_i \varphi_i \mid a_i \in \mathbb{R}\}$ with $\varphi_i(x) = \kappa(x_i, x)$

$$\left(\sum_{i=1}^q a_i \varphi_i \right)^\top \left(\sum_{j=1}^q b_j \varphi_j \right) \stackrel{\text{def}}{=} \sum_{i,j=1}^q a_i b_j \kappa(x_i, x_j) \Rightarrow F(x) = \begin{cases} 1 & \text{if } \sum_{i \in S} \hat{a}_i \kappa(x, x_i) + \hat{b} \geq 0 \\ -1 & \text{if } \sum_{i \in S} \hat{a}_i \kappa(x, x_i) + \hat{b} < 0 \end{cases}$$



Naïve Bayes classifier

- ◆ Classifier based on the Bayes theorem (C - class, x - input):

$$P(C|x) = \frac{P(x|C)P(C)}{P(x)}$$

- ◆ Computing $P(x|C)$ is infeasible $\Rightarrow$ *naïve* Bayes assumption:

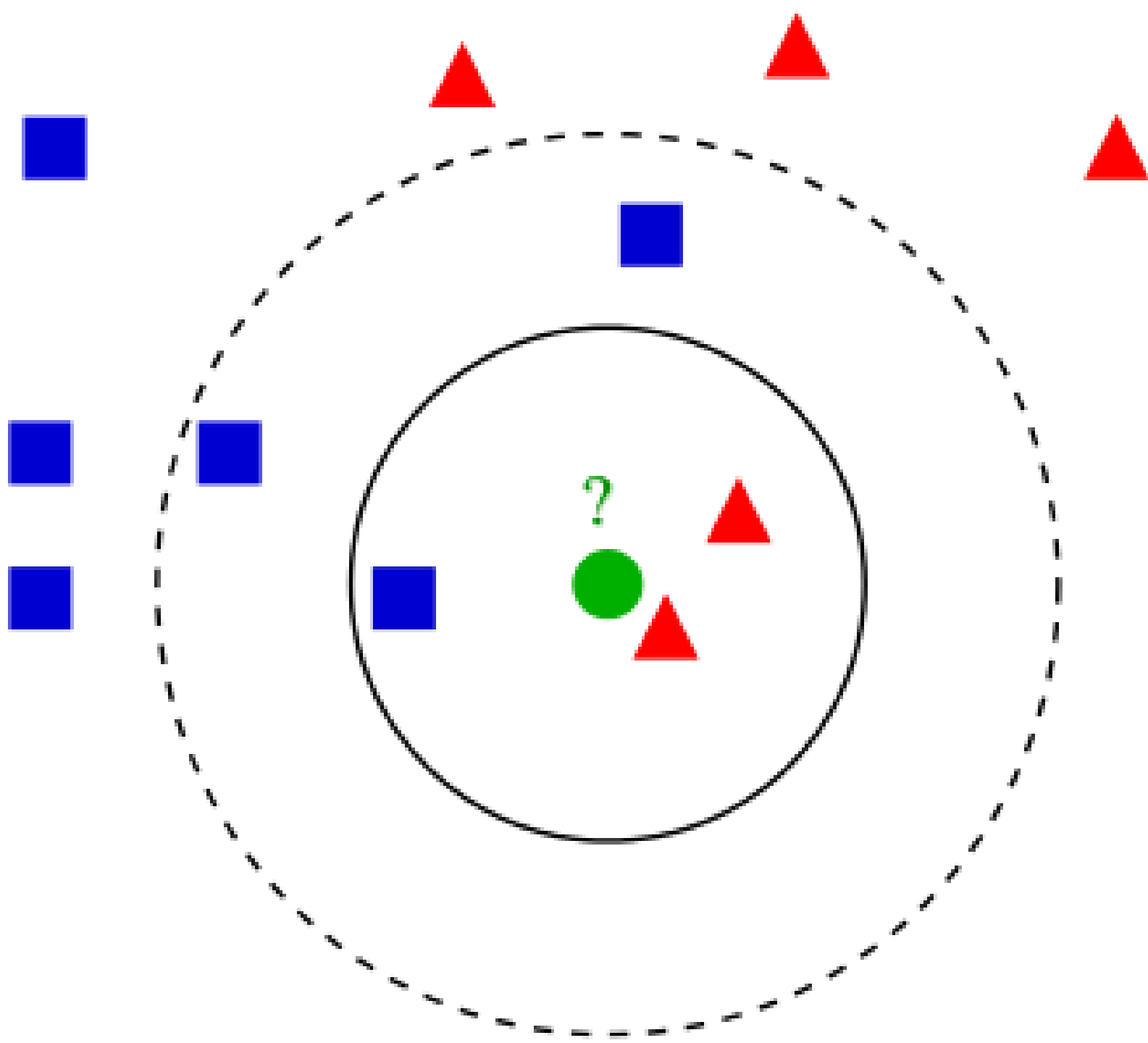
$$P(x|C) = \prod_{i=1}^d P(x_i|C)$$

- ◆ Estimation of $(x_i|C)$: multimodal | Bernoulli | Gaussian

Nearest neighbours learning

- ◆ Instance-based supervised, searches the k nearest neighbours set

$N_k(x) = \{i_1, \dots, i_k\}$, where $\|x - x_{i_j}\|$ are among the k smallest



Nearest neighbours learning

- ◆ Instance-based supervised, searches the k nearest neighbours set

$N_k(x) = \{i_1, \dots, i_k\}$, where $\|x - x_{i_j}\|$ are among the k smallest

- ◆ *Classification*: predicted class is $\hat{c}(x) = \arg \max_c \#\{x_i \in N_k(x) | y_i = c\}$

- ◆ *Regression*: predicted value is the average $\hat{f}(x) = \frac{1}{k} \sum_{i=1}^k y_i$

- ◆ k influences bias-variance decomposition for $\hat{f}$ estimating a function f

$$\mathbb{E} \left[\left(\hat{f}(x) - f(x) \right)^2 \right] = \underbrace{\left(\mathbb{E} \hat{f}(x) - f(x) \right)^2}_{\text{bias}^2} + \underbrace{\mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E} \hat{f}(x) \right)^2 \right]}_{\text{variance}} + \sigma^2$$

Rule learning

- ◆ Represents knowledge in *symbolic form* as

implications in propositional logic, aka if-then-rules:

$$\underbrace{(C_1 \wedge C_2 \wedge \cdots \wedge C_m)}_{\text{antecedent}} \Rightarrow \underbrace{H}_{\text{consequent, aka hypothesis}}$$

- C_j – literal defining an attribute-value condition

- ◆ *Interpretable* $\Rightarrow$ valuable in domains like medicine, law, finance
- ◆ Main learning algorithms: separate-and-conquer, association rules

Ensemble learning

- ◆ Combining multiple base learners – *weak learners* $h_1, \dots, h_m: X \rightarrow Y$,
to form a stronger predictive model – *strong* learner $H: X \rightarrow Y$

- ◆ *Regression* ensemble typically predicts a weighted combination:

$$H(x) = \sum_{i=1}^m w_i h_i(x), w_i > 0, \sum_{i=1}^m w_i = 1$$

- ◆ *Classification* ensemble typically predicts the majority vote:

$$H(x) = \arg \max_{y \in Y} \# \{i \mid 1 \leq i \leq m \text{ \& } h_i(x) = y\}$$

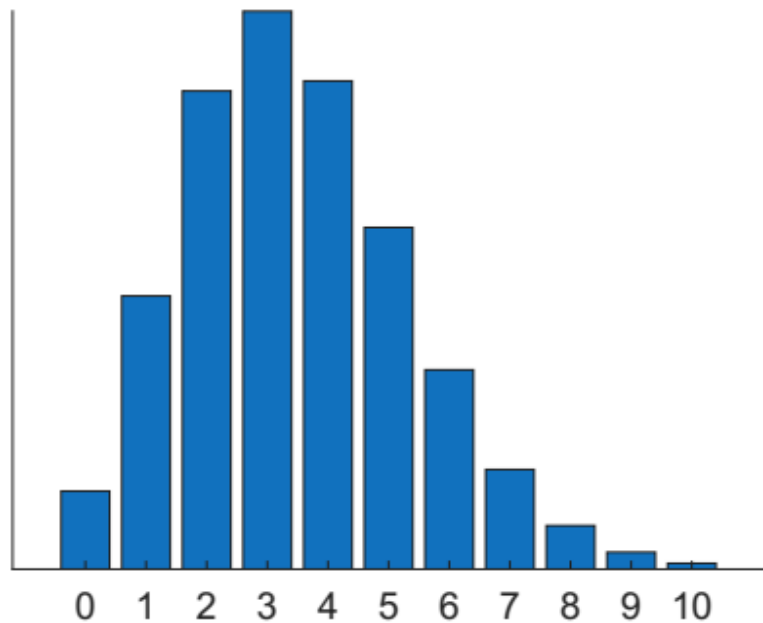
Random Variables

- ◆ Functions quantifying the outcomes of probabilistic phenomena by real numbers $\Rightarrow$ allow applying analysis + measure theory
- ◆ *Probability space* $(\Omega, \mathcal{E}, P)$: Ω – sample space, $\mathcal{E}$ – σ -algebra of subsets of Ω , called events, $P: \mathcal{E} \rightarrow [0,1]$ – probability
- ◆ *Borel sets* $\mathcal{B}_n$: the smallest σ -algebra containing open $\mathbb{R}^n$ sets
- ◆ $X: \Omega \rightarrow \mathbb{R}^n$ is called random variable if $\forall B \in \mathcal{B}_n: X^{-1}(B) \in \mathcal{E}$

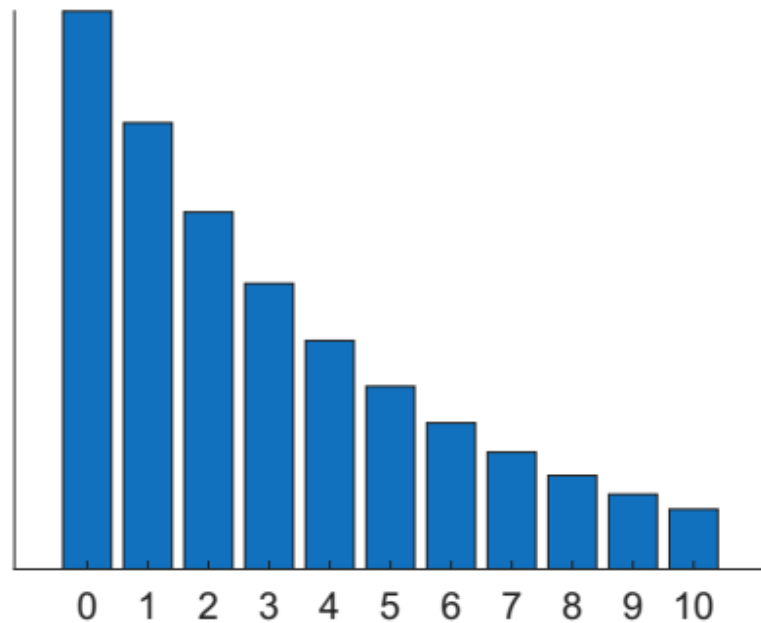
Distribution and Moments

- ◆ Cumulative distribution function (*CDF*) of X : $F(x) = P(X \leq x)$
 - X called *discrete* for a step function $F \Rightarrow X(\Omega)$ countable

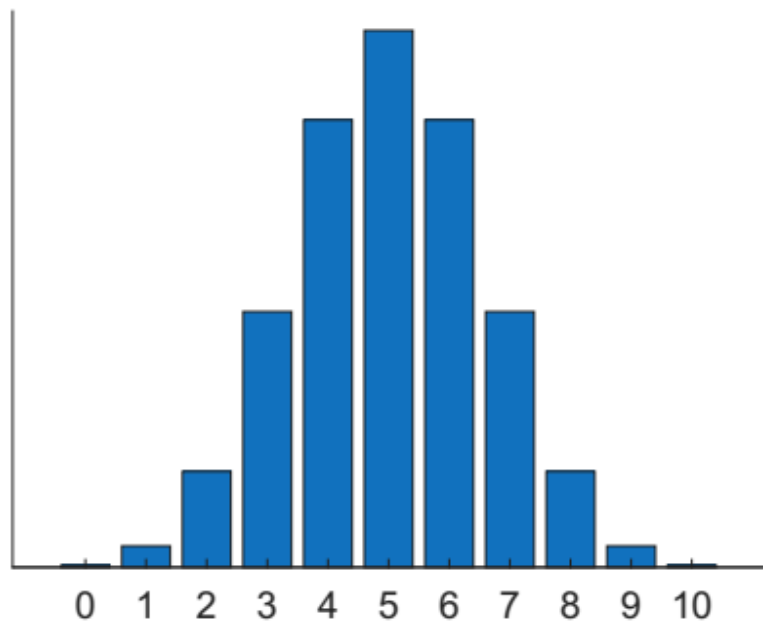
Poisson distribution



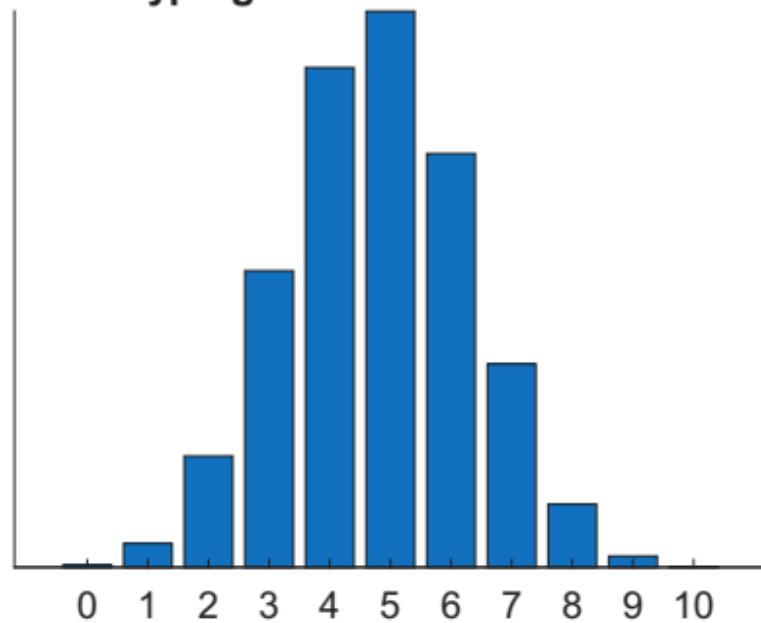
Geometric distribution



Binomial distribution



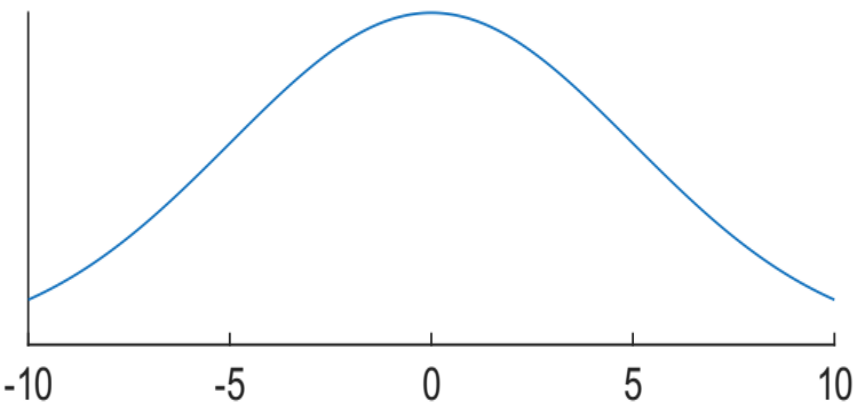
Hypergeometric distribution



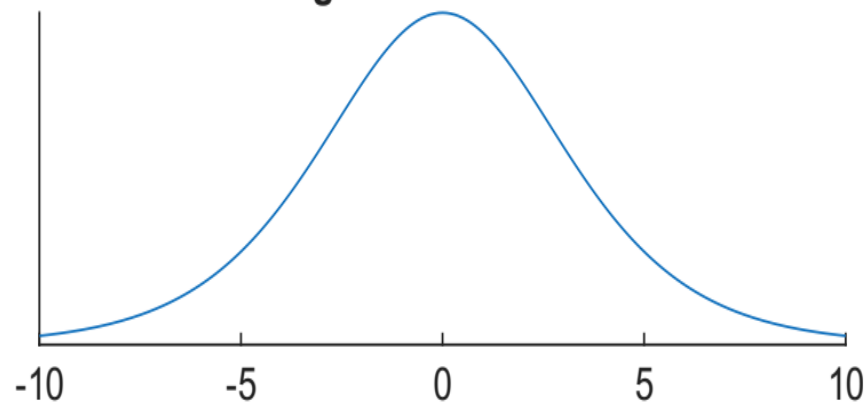
Distribution and Moments

- ◆ Cumulative distribution function (*CDF*) of X : $F(x) = P(X \leq x)$
 - X called *discrete* for a step function $F \Rightarrow X(\Omega)$ countable
 - X called *continuous* for F differentiable, $f = F'$ is called density

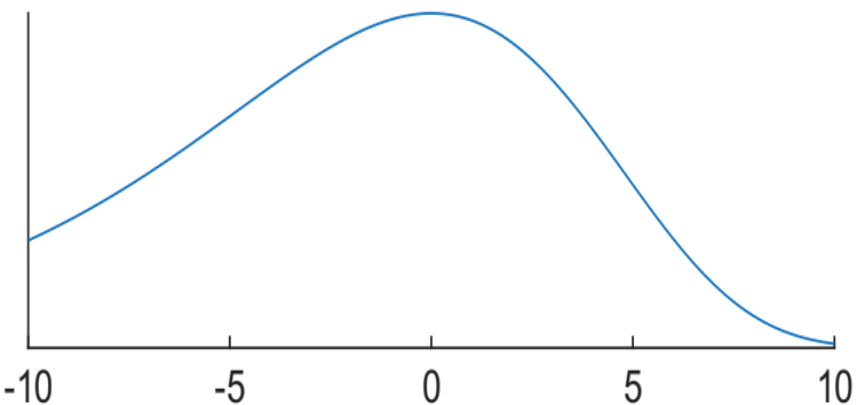
Normal distribution



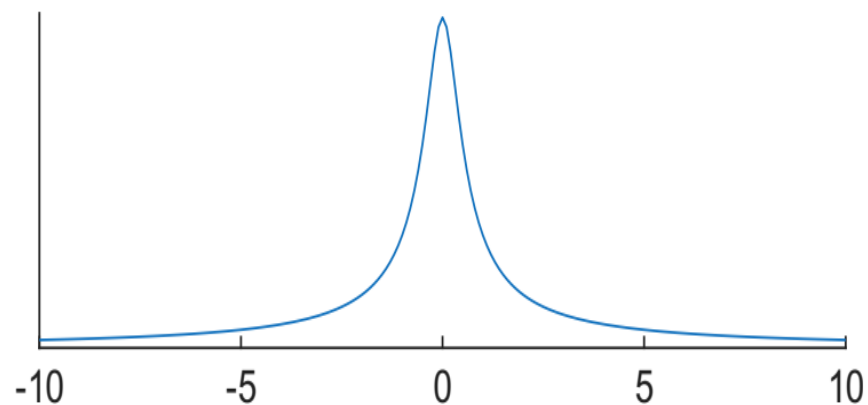
Logistic distribution



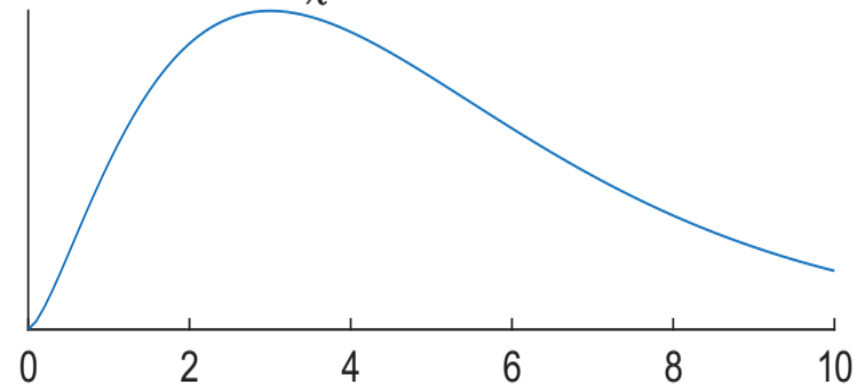
Extreme values distribution



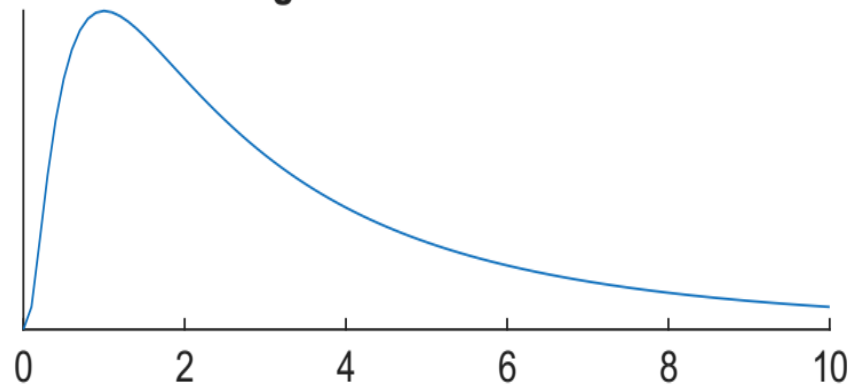
Student t distribution



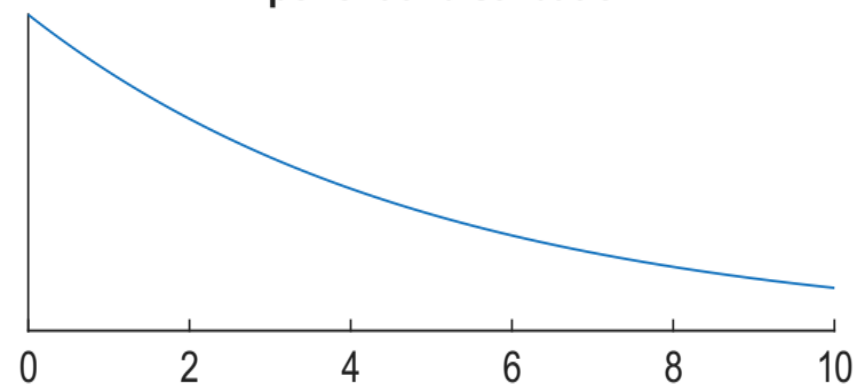
χ^2 distribution



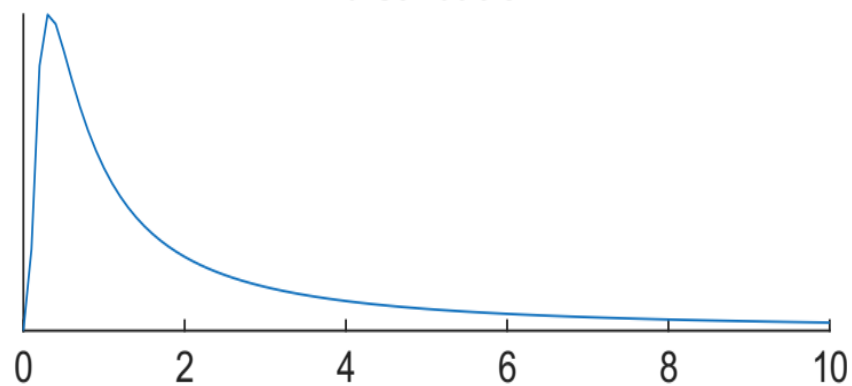
Lognormal distribution



Exponential distribution



F distribution



Distribution and Moments

- ◆ Cumulative distribution function (*CDF*) of X : $F(x) = P(X \leq x)$
 - X called *discrete* for a step function $F \Rightarrow X(\Omega)$ countable
 - X called *continuous* for F differentiable, $f = F'$ is called density
- ◆ r -th raw moment of X : $\mu'_r = \mathbb{E}[X^r]$, in particular $\mu'_1 = \mu(X) = \mathbb{E}[X]$
- ◆ r -th central moment (about $\mathbb{E}[X]$) of X : $\mu_r = \mathbb{E}[(X - \mathbb{E}[X])^r]$
 - variance: $\sigma^2 = \text{var}(X) = \mu_2$, skewness: $\gamma_1 = \frac{\mu_3}{\sigma^3}$, kurtosis: $\gamma_2 = \frac{\mu_4}{\sigma^4}$

Conditional Distribution and Moments

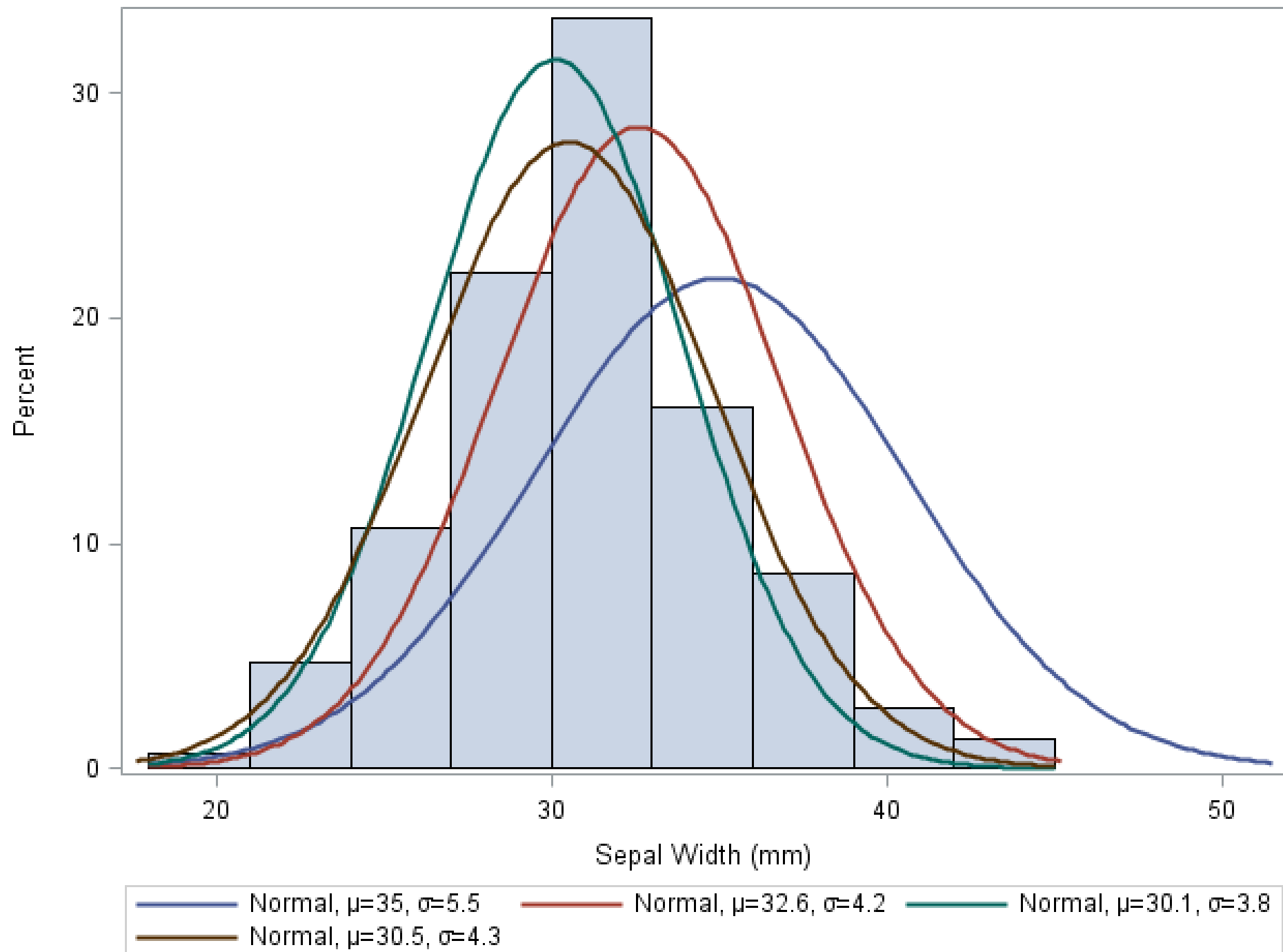
- ◆ Describe the distribution of Y conditioned on X ($X = x$)
- ◆ Conditional expectation $\mathbb{E}[Y|X]$ of Y conditioned on X : random variable *measurable* in the *σ -algebra Σ_X generated by X*
- ◆ For A from both Σ_X and the σ -algebra Σ_Y generated by Y :

$$\int_A \mathbb{E}[Y|X](\omega) dP(\omega) = \int_A Y(\omega) dP(\omega)$$

- ◆ Among functions g measurable in Σ_X : $\mathbb{E}[Y|X]$ minimizes $\mathbb{E}[(Y - g(x))^2]$

Statistical Estimating

- ◆ Deduces probability distribution properties from a finite sample
 - example properties: expectation, variance, regression coefficients
- ◆ *Point* estimate: single best guess of the property
- ◆ *Interval* estimate: contains the value with required probability
- ◆ *Moments* method: θ_m such that for $k = 1, \dots, n$: $\frac{1}{n} \sum_{i=1}^n X_i^k = \mathbb{E}[X_{\theta_m}^k]$
- ◆ *Maximum likelihood* estimate: $\theta_{mle} = \arg \max_{\theta} L(\theta)$ with $L(\theta) = \prod_{i=1}^n f_{\theta}(X_i)$



Statistical Hypothesis Testing

◆ Procedure deciding whether a hypothesis H_0 about a distribution should be rejected in favour of its alternative H_1

◆ Is based on the stochastic *proof by contradiction*:

$$(H_0 \text{ true} \rightarrow \text{high } P(\text{data}|H_0)) \Leftrightarrow (\text{low } P(\text{data}|H_0) \rightarrow H_0 \text{ false})$$

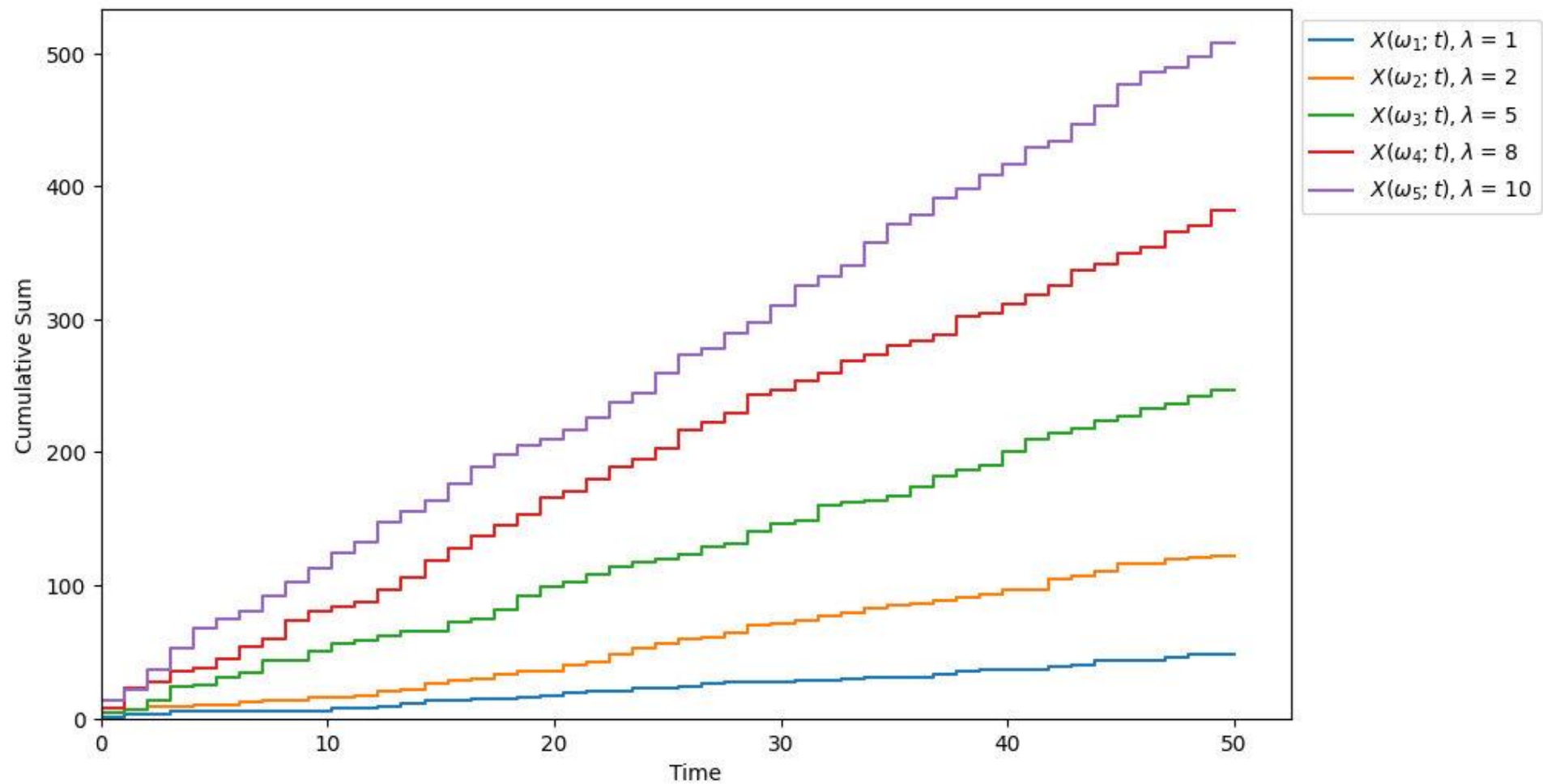
◆ Test statistic: random variable on $(X_1, \dots, X_n)$, assumed distribution

$$\text{p-value} = \begin{cases} P(T \geq T(X_1, \dots, X_n)) & \text{for } T \text{ with a right-tailed distribution} \\ P(|T| \geq |T(X_1, \dots, X_n)|) & \text{for } T \text{ with a 2-tailed symmetric distribution} \end{cases}$$

Random Process

- ◆ Random variables collection $\{X(t)|t \in T\}$, indexed by a $T \subset \mathbb{R}/\subset \mathbb{R}^k$
- ◆ *Mean* $m_X(t) = \mathbb{E}[X(t)]$, *autocorrelation* $R_X(t_1, t_2) = \mathbb{E}[X(t_1)X(t_2)]$
- ◆ Wide sense *stationarity* of X : 1. constant m_X , 2. $R_X(t_1, t_2) = R_X(t_2 - t_1)$
- ◆ Discrete | continuous T , discrete | continuous states ($\sim$ values)
 - discrete T example: *Poisson* process $\{N(t)|t \in \mathbb{N}\}$, counts random arrivals
 - continuous $T \subset \mathbb{R}^k$ example: *Gaussian* process (finite $T' \subset T \Rightarrow$ normal)

Poisson Process with $X(\omega_i; t)$



Bayesian Approach to Randomness

- ◆ Bayesian statistics interprets probability as a *belief degree*
 - whereas frequentist interpretation: probability = long-run relative frequency
 - belief degree is updated as new information arrives
- ◆ Relies on the Bayes theorem: $P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$
 - $P(\theta)$ – prior, $P(D|\theta)$ – likelihood of data, $P(\theta|D)$ – posterior
- ◆ $\Rightarrow$ Bayesian statistics relies on *posterior $\propto$ likelihood * prior*

Aleatoric and Epistemic Uncertainty

- ◆ Aleatoric uncertainty: *randomness* conveyed by random variables
 - *inherent* in the systems/processes described by them
- ◆ Epistemic uncertainty: arises from *lack of knowledge*
 - 1. insufficient data, 2. incomplete models (differing from reality)
- ◆ Aleatoric irreducible, epistemic reducible by more knowledge:
 - more data, better generalizing models, additional expert judgement